

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Grado en Ingeniería y Sistemas de Datos

TRABAJO DE FIN DE GRADO

Título: Aplicación de técnicas de Machine Learning para la identificación y optimización de patrones en 'Swing Chart' en mercados financieros

Autor: D. Lijun Zhou

Supervisión técnica:

Supervisión académica: Dr. Eduardo López Gonzalo

Departamento: Departamento de Señales, Sistemas y Radiocomunicaciones.

MIEMBROS DEL TRIBUNAL

Presidente:

Vocal:

Secretario:

Suplente:

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

.....

Fecha de lectura:

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Resumen

El presente proyecto tiene como objetivo el desarrollo e implementación de un sistema basado en técnicas de Machine Learning orientado al análisis y detección de patrones en “Swing Charts”, una herramienta ampliamente utilizada en el análisis técnico de los mercados financieros. A partir de datos históricos y en tiempo real, el sistema generará representaciones de “Swing Charts” a partir de “Bar Charts” y aplicará algoritmos de aprendizaje supervisado y no supervisado para identificar configuraciones recurrentes, como dobles máximos, dobles mínimos o rupturas de tendencia.

El desarrollo se llevará a cabo principalmente en Python, utilizando librerías como pandas, scikit-learn y matplotlib, además de entornos de conexión con plataformas de trading como Interactive Brokers API (IBKR). El proyecto busca evaluar la capacidad predictiva de los modelos entrenados y su utilidad en la generación de señales de compra o venta dentro de una estrategia de trading algorítmico. Con ello, se pretende aportar un enfoque innovador que combine la solidez del análisis técnico clásico con la adaptabilidad de las técnicas modernas de inteligencia artificial.

Palabras clave

Trading algorítmico, predicción de mercados financieros, automatización, análisis en tiempo real, backtesting.

Summary

XXXXXXXXX

Keywords

XXXXXXXXX

Índice

1	Introducción y objetivos	1
1.1	Introducción	1
1.2	Objetivos	2
2	Estado del arte	5
2.1	Tendencias en el trading algorítmico	5
2.2	Modelos para series temporales: RNN, LSTM y GRU	5
2.2.1	LSTM	5
2.2.2	GRU	6
2.3	Tendencias recientes en el trading cuantitativo	7
2.4	Comparativa entre indicadores tradicionales, ML clásico, ML profundo y IA agéntica	7
3	Desarrollo	9
3.1	Pipeline	9
3.2	Desarrollo del autómata	11
3.2.1	Obtención de datos de los futuros	12
3.2.2	Carga y limpieza	13
3.2.3	Cálculo de características	16

3.2.4	Selección de características	17
3.2.5	Definición y cálculo de la clase objetivo	18
3.2.6	Aprendizaje supervisado	19
3.2.7	Backtest de las predicciones	24
3.2.8	Simulación de Monte Carlo	26
3.2.9	Simulación de Monte Carlo	26
3.2.10	Gestión de cartera	27
3.2.11	Manejo del riesgo	28
3.2.12	Operación	29
3.2.13	Logging	30
3.3	Despliegue usando contenedores de Docker	31
3.3.1	Configuración del contenedor principal	31
3.3.2	Bases de datos	32
3.3.3	Visualización	34
3.3.4	Alertas y avisos	35
4	Resultados	36
4.1	Resultados	36
4.1.1	Análisis de resultados de entrenamiento	40
4.1.2	Análisis de resultados de optimización, Backtest de la cartera y Simulación de Monte Carlo	42
5	Conclusiones y líneas futuras	44
5.1	Conclusiones	44
5.2	Líneas futuras	44
5.2.1	Aprendizaje no supervisado	47

5.2.2	Pyomo	47
5.2.3	Orquestación y MLOps	48
6	Aspectos éticos, económicos, sociales y ambientales	50
6.1	Introducción	50
6.2	Descripción de impactos relevantes relacionados con el proyecto	50
6.3	Análisis detallado de alguno de los principales impactos	50
6.4	Conclusiones	50
7	Presupuesto económico	51
	Bibliografía	52
	Anexos	53
A	Resultados de los entrenamientos	53
B	Resultados de los backtest individuales	53
C	Repositorio de GitHub con el código	53

Listado de figuras

2.1	Arquitectura LSTM, [1]	6
2.2	Arquitectura GRU, [1]	6
3.1	Información estática y dinámica del contrato de futuros AD obtenida mediante la API de Interactive Brokers.	13
3.2	Comparación antes y después de ajustar	15
3.3	Añadir img de walk forward	21
3.4	Ejemplo de registro JSON generado por el sistema para cada operación ejecutada.	30
3.5	Ejemplo de un <i>Dashboard</i> en Grafana	35
3.6	Ejemplo con alertas recibidas en Slack	35
4.1	Añadir img con equity curve, max drawdown y sharpe ratio	36
4.2	Añadir img comparativa del Backtest vs Robotrader	37
4.3	Añadir img comparativa del Monte Carlo vs Robotrader	37
4.4	Añadir img comparativa del Monte Carlo vs Robotrader	37
4.5	Añadir img comparativa del Monte Carlo vs Robotrader	38
4.6	Resultados obtenidos por el modelo LSTM.	40
4.7	Resultados obtenidos por el modelo GRU.	41
4.8	Añadir img con evolución de las métricas dependiendo de los umbrales	42

4.9	Añadir img con una simulación de Monte Carlo con los percentiles	42
5.1	Añadir img con algunos plots de las características más usadas como modelos solos	46
5.2	Añadir img con algunos plots de las características más usadas como modelos solos	46

Listado de tablas

1.1	Características principales de los contratos de futuros analizados	4
3.1	Ejemplo de registro histórico OHLCV de un contrato de futuros	14
3.2	Criterios de selección de predicciones válidas en la Fase 1 del backtest.	24
3.3	Modalidades operativas utilizadas en la Fase 2 del backtest.	25
3.4	Estructura de la tabla metrics	32
3.5	Estructura de la tabla risk_events	32
3.6	Estructura de la tabla signals	33
3.7	Estructura de la tabla trades	33
4.1	Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-01-20 hasta 2026-06-01	39
4.2	Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-03-27 hasta 2026-06-01, aunque Robotrader empezó el 6 para tener datos para las primeras predicciones	39

Todo list

☐	No olvidar	22
☐	Añadir grafica de la evolución del equity, comparativa contra Robotrader	38
☐	Añadir Monte Carlo	38
☐	Añadir curva auc roc para las dos capas de clasificación	41
☐	Añadir curva loss por fold	41
☐	Preguntar si demasiadas lineas futuras demuestra poco o mucho interes en el trabajo	44
☐	Explicar qué es Pyomo	47
☐	Tanto Airflow como MLflow, ver si hay tiempo para implementarlos	48

Glosario

OHLCV - Precios Open High Low y Close y Volumen

XXXX - XXXXXXXXX

Introducción y objetivos

1.1 Introducción

El trading financiero tradicional presenta una limitación fundamental, una fuerte dependencia de los criterios subjetivos, los sesgos cognitivos y el estado emocional de cada inversor. Factores como el miedo o la euforia pueden conducir a decisiones inconsistentes y alejadas de una gestión óptima del riesgo. Para mitigar este problema surgió el *trading algorítmico*, un enfoque donde las decisiones se toman en base a reglas matemáticas predefinidas, eliminando en gran medida la intervención humana durante la operativa.

La evolución natural del *trading algorítmico* ha sido la incorporación de modelos de aprendizaje automático (ML) que se entrenan usando un conjunto de características generadas a partir de datos históricos de los diferentes activos financieros a operar. Estos modelos permiten aprender patrones complejos a partir de datos históricos, capturar relaciones no lineales y adaptarse a diferentes regímenes de mercado. El presente trabajo se centra en esta línea, el diseño y desarrollo de un sistema de trading basado en ML capaz de operar de forma autónoma sobre un conjunto diverso de futuros financieros.

Para la elaboración de este trabajo se toman como referencia dos obras. Por un lado, *Trading Algorítmico con Python* de Isaac Trullàs [2], que proporciona una guía sobre el flujo de trabajo completo para el desarrollo de una estrategia algorítmica, desde la carga y limpieza de los datos, pasando por el cálculo y validación de las características que alimentarán a los modelos hasta la evaluación final mediante el cálculo de métricas para comprobar que la estrategia desarrollada es consistente, robusta y capaz de generar beneficios de forma sostenida. Por otro lado, *Safety in the Market* de David E. Bowden[3], que introduce conceptos fundamentales de análisis de mercado y del trading.

En los mercados financieros existen diferentes tipos de activos, cada uno con sus características, con diferentes niveles de riesgo y con comportamientos distintos frente a las condiciones económicas. Entre los más comunes se encuentran las acciones, que representan participaciones dentro de una empresa; los bonos, que constituyen instrumento de deuda emitidos por gobiernos o corporaciones; las materias primas, como el oro, petróleo o productos agrícolas; y las divisas, cuyo valor se basa en el tipo de cambio que existe entre monedas. Por otra parte existen derivados financieros, cuyo valor depende de un activo subyacente. Dentro de esta categoría se encuentran los futuros, contratos estandarizados que permiten tomar posiciones largas o cortas sobre índices bursátiles, divisas, materias primas o renta fija. Una posición larga significa comprar barato esperando que el precio aumente para vender, mientras que una posición corta consiste en vender anticipadamente esperando que el precio disminuya para recomprar más barato. Existe una asimetría en el riesgo asociado a cada tipo de posición, la

ventaja de las posiciones a largo reside en que en caso de que el precio disminuya, solo puede caer hasta 0 mientras que en las posiciones cortas el precio puede aumentar sin un techo, haciendo especialmente relevante la gestión de riesgo.

En este trabajo se ha optado la operación sobre futuros en vez de sobre otros activos por diversas razones.

- **Mercados regulados y centralizados:** Las negociaciones se realizan en mercados regulados y centralizados (CME, MEFF, entre otros), garantizando transparencia y control, disminuyendo los riesgos de manipulación.
- **Alta liquidez:** Ofrecen una elevada liquidez, facilitando la ejecución eficiente de órdenes incluso en estrategias algorítmicas.
- **Estandarización de los contratos:** Todos los contratos comparten especificaciones fijas —tamaño, tick, vencimiento, márgenes— lo que simplifica la obtención, comparación y tratamiento de los datos.
- **Uso de márgenes y apalancamiento:** El uso de márgenes permite operar con apalancamiento, aumentando el potencial de beneficio aunque también el riesgo. Por ello, la gestión del riesgo es fundamental, como se detallará más adelante. Este mecanismo permite controlar posiciones mayores con un capital limitado.
- **Rollover:** Existe la posibilidad de realizar *rollover*, evitando la entrega física del activo y permitiendo mantener la operativa de forma continua.

1.2 Objetivos

El objetivo principal de este presente trabajo es desarrollar un sistema capaz de operar de manera autónoma una cartera con un valor de un millón de dólares, maximizando el beneficio mientras asume el menor riesgo posible. Como se puede observar en la Tabla 1.1, el sistema operará sobre 13 futuros distintos que pertenecen a distintas categorías, divisas, materias primas, índices bursátiles y bonos del Tesoro estadounidense, proporcionando así una exposición diversificada a diferentes clases de activos y dinámicas de mercado.

A continuación se describen los futuros sobre los que se llevará a cabo la operativa.

AD - Australian Dollar

Futuro sobre el dólar australiano frente al dólar estadounidense. Muy utilizado en estrategias macro y ligadas a materias primas.

BP - British Pound

Futuro sobre la libra esterlina. Alta liquidez y relevancia en estrategias FX y cobertura frente a decisiones del Banco de Inglaterra.

CL - Crude Oil WTI

Futuro del petróleo WTI. Uno de los contratos más negociados del mundo, con elevada volatilidad y fuerte sensibilidad geopolítica.

EC - Euro FX

Futuro del euro frente al dólar. El contrato FX más líquido del CME, adecuado para estrategias direccionales y de reversión.

ES - E-mini S&P 500

Futuro del índice S&P 500 en formato reducido. Altísima liquidez y representatividad del mercado estadounidense.

GC - Gold

Futuro del oro. Activo refugio con dinámicas propias en periodos de incertidumbre y movimientos de tipos reales.

MFXI - Micro Euro FX (M6E)

Versión micro del futuro del euro. Permite exposición granular con menor margen, ideal para calibración fina.

NG - Natural Gas

Futuro del gas natural. Muy volátil y con fuerte estacionalidad, relevante en estrategias de energía.

NQ - E-mini Nasdaq 100

Futuro del Nasdaq 100. Más volátil que el ES, con fuerte exposición al sector tecnológico.

YM - E-mini Dow Jones

Futuro del Dow Jones Industrial Average. Menor volatilidad y comportamiento más estable que NQ y ES.

ZB - 30Y US Treasury Bond

Futuro del bono del Tesoro a 30 años. Muy sensible a expectativas macroeconómicas de largo plazo.

ZN - 10Y US Treasury Note

Futuro del Treasury a 10 años. El contrato de renta fija más líquido del mundo, referencia clave en estrategias macro.

ZS - Soybean Futures

Futuro de la soja. Presenta estacionalidad marcada y sensibilidad a factores climáticos y agrícolas globales.

Tabla 1.1: Características principales de los contratos de futuros analizados

Símbolo	Nombre	Mult.	Tick Size	Tick Value	Margen Init
AD	Australian Dollar	100000	0.00005	\$5	\$2503.59
BP	British Pound	62500	0.0001	\$6.25	\$2585.50
CL	Crude Oil WTI	1000	0.01	\$10	\$28336.53
EC	Euro FX	125000	0.00005	\$6.25	\$3671.13
ES	E-mini S&P 500	50	0.25	\$12.5	\$32380.88
GC	Gold	100	0.1	\$10	\$44136.61
MFXI	Micro Euro FX (M6E)	12500	0.0001	\$1.25	\$367.60
NG	Natural Gas	10000	0.001	\$10	\$6592.86
NQ	E-mini Nasdaq 100	20	0.25	\$5	\$60368.33
YM	E-mini Dow Jones	5	1	\$5	\$17296.56
ZB	30Y US Treasury Bond	1000	0.03125	\$31.25	\$4259.66
ZN	10Y US Treasury Note	1000	0.015625	\$15.625	\$2156.85
ZS	Soybean Futures	5000	0.0025	\$12.5	\$4077.58

Para alcanzar este objetivo se ha diseñado una arquitectura basada en modelos de aprendizaje automático que asisten en la toma de decisiones operativas. La propuesta inicial contempla el uso combinado de modelos de aprendizaje supervisado y no supervisado, cuyos papeles se detallará en los siguientes capítulos. En términos generales, los modelos de aprendizaje profundo se emplearán como modelos supervisados para la generación de señales de trading, mientras que los modelos no supervisados se utilizarán para identificar regímenes de mercado y agrupar comportamientos similares mediante técnicas de clustering.

La arquitectura propuesta se concibe, por tanto, como un sistema híbrido: los modelos supervisados proporcionan señales direccionales, y los modelos no supervisados aportan información contextual que puede mejorar la robustez y la interpretación del entorno de mercado.

No obstante, aunque esta sea la intención inicial, la implementación final puede diferir debido a limitaciones encontradas durante el desarrollo, cuyas motivaciones serán explicadas en los capítulos correspondientes. En cualquier caso, la arquitectura mantiene la capacidad de integrar modelos no supervisados en futuras ampliaciones.

Finalmente, el objetivo último de este trabajo es poder poner el uso la mayor cantidad de conocimiento obtenido de estos años de estudio en un solo trabajo, incluyendo principalmente programación en Python, despliegue del trabajo en Docker, entrenamiento de modelos de Machine Learning, optimización y diseño de pipelines de datos.

Estado del arte

2.1 Tendencias en el trading algorítmico

El trading algorítmico ha experimentado una evolución significativa en las últimas décadas, pasando de estrategias basadas exclusivamente en indicadores técnicos a sistemas que integran técnicas avanzadas de aprendizaje automático y modelos de aprendizaje profundo. El estado del arte actual combina ingeniería de datos, ingeniería de características, modelado predictivo, análisis cuantitativo y técnicas de optimización, con el objetivo de capturar patrones complejos en series temporales financieras altamente ruidosas y no estacionarias.

2.2 Modelos para series temporales: RNN, LSTM y GRU

Las Redes Neuronales Recurrentes (RNN) clásicas [4] fueron diseñadas para modelar dependencias temporales, pero presentan el problema del desvanecimiento del gradiente. Durante la retropropagación, los gradientes se atenúan al multiplicarse repetidamente por derivadas de funciones de activación como *sigmoid* o *tanh*, lo que provoca que las capas iniciales reciban gradientes casi nulos y solo se aprendan dependencias de corto plazo.

Para resolver esta limitación se propusieron dos arquitecturas mejoradas: LSTM (Long Short-Term Memory) [5] y GRU (Gated Recurrent Unit) [6]. Ambas introducen mecanismos de compuertas que regulan el flujo de información y permiten capturar dependencias de largo plazo de manera estable.

2.2.1 LSTM

Las LSTM introducen una memoria interna (*cell state*) y tres compuertas (entrada, olvido y salida) que controlan qué información se mantiene y cuál se descarta, permitiendo modelar dinámicas temporales complejas.

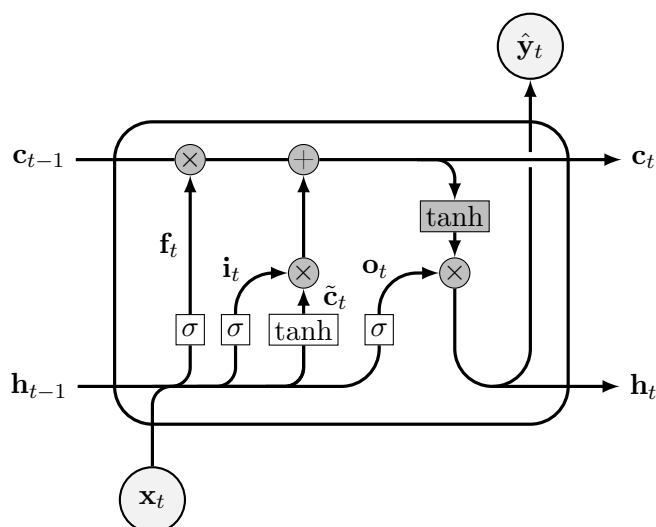


Figura 2.1: Arquitectura LSTM, [1]

2.2.2 GRU

Las GRU simplifican esta estructura mediante dos compuertas (actualización y reinicio), reduciendo el número de parámetros y acelerando el entrenamiento, manteniendo un rendimiento comparable al de las LSTM.

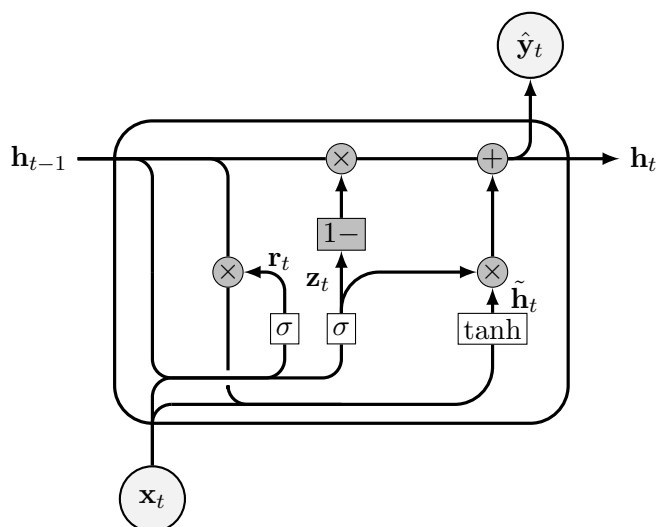


Figura 2.2: Arquitectura GRU, [1]

Ambas arquitecturas han demostrado ser especialmente útiles en series temporales financieras

debido a su capacidad para modelar dinámicas no lineales y dependencias temporales complejas y patrones ruidosos característicos de los mercados.

A pesar de su capacidad para capturar relaciones no lineales y dependencias temporales complejas, los modelos de aprendizaje profundo presentan un problema fundamental: su falta de interpretabilidad. Estas arquitecturas actúan como cajas negras, ya que no ofrecen una explicación clara de los factores que conducen a una predicción concreta. Esta opacidad dificulta comprender qué patrones del mercado está utilizando el modelo.

2.3 Tendencias recientes en el trading cuantitativo

Las tendencias recientes en el trading cuantitativo incluyen

- **Modelos basados en Transformers** para series temporales, capaces de capturar dependencias largas mediante mecanismos de atención.
- **Meta-learning** y técnicas de adaptación rápida a nuevos activos o regímenes de mercado.
- **Modelos híbridos** que combinan indicadores tradicionales con aprendizaje profundo.
- **Detección de regímenes de mercado** mediante clustering no supervisado para contextualizar las señales de trading.
- **Estudios de ablation** para evaluar la contribución real de cada componente del sistema y evitar sobreajuste.

2.4 Comparativa entre indicadores tradicionales, ML clásico, ML profundo y IA agéntica

Los enfoques utilizados en trading cuantitativo han evolucionado de manera significativa en las últimas décadas. Tradicionalmente, las estrategias se basaban en indicadores técnicos, como medias móviles, RSI, MACD o ATR, que transforman el precio en señales determinísticas. Aunque estos indicadores son sencillos de interpretar y computacionalmente eficientes, presentan limitaciones importantes: dependen de parámetros fijos, no capturan relaciones no lineales y suelen fallar en entornos de mercado cambiantes o altamente volátiles.

El siguiente paso en esta evolución fue la incorporación de modelos de aprendizaje automático clásico, como árboles de decisión, Random Forest, SVM o regresiones penalizadas. Estos modelos permiten aprender relaciones más complejas entre múltiples variables y generalizan mejor

que los indicadores tradicionales. Sin embargo, siguen teniendo dificultades para capturar dependencias temporales profundas y requieren un diseño manual de características (feature engineering) para representar adecuadamente la dinámica del mercado.

Con el auge del cómputo acelerado y la disponibilidad de grandes volúmenes de datos, surgieron los modelos de aprendizaje profundo, especialmente arquitecturas como LSTM, GRU, CNN 1D y, más recientemente, Transformers. Estos modelos son capaces de aprender patrones no lineales, dependencias de largo plazo y estructuras temporales complejas directamente a partir de los datos, reduciendo la necesidad de ingeniería manual. Su principal desventaja es la falta de interpretabilidad, ya que funcionan como cajas negras, y su tendencia al sobreajuste si no se aplican técnicas adecuadas de regularización y validación temporal.

Finalmente, en los últimos años ha emergido el concepto de IA agentica aplicada al trading, donde agentes autónomos combinan modelos predictivos, razonamiento secuencial y toma de decisiones basada en objetivos. Aunque este enfoque es prometedor, su uso en entornos financieros reales es todavía limitado debido a la complejidad de su validación, la dificultad para garantizar estabilidad y la necesidad de mecanismos robustos de control del riesgo.

Desarrollo

3.1 Pipeline

El desarrollo del sistema se estructura en varias fases dentro del proceso ETL (Extract, Transform, Load). En primer lugar, se realiza la carga de los datos históricos de los distintos contratos de futuros que se van a operar. A continuación, estos datos se someten a un proceso de limpieza y filtrado para eliminar valores erróneos, inconsistencias y posibles fuentes de ruido. Posteriormente, se lleva a cabo la transformación de los datos en un conjunto de características que servirán como entrada para los modelos de aprendizaje automático. Una vez entrenados los modelos, se evalúa su rendimiento mediante diferentes métricas y utilizando datos históricos no vistos para comprobar su capacidad de generalización. Finalmente, el sistema entra en fase operativa, donde procesa datos en tiempo real y genera señales de trading de manera autónoma.

El flujo de trabajo se organiza siguiendo la arquitectura medallón, donde los datos atraviesan por tres capas diferenciadas:

Bronce :

Contiene los datos históricos y en vivo en su forma cruda, tal y como se obtienen de las fuentes originales.

Plata :

En esta capa los datos se limpian, se validan, se filtran y se normalizan, garantizando su coherencia y calidad antes de ser utilizados.

Oro :

Los datos se transforman en *features* y *targets* listos para el entrenamiento de los modelos supervisados y para la inferencia en tiempo real.

A continuación muestro un mapa conceptual de las diferentes etapas y los hitos más importantes que se van a desarrollar en este trabajo.

1. **Carga y estandarización de datos:** descarga de datos minuto a minuto y unificación temporal a UTC.
2. **Limpieza y validación:** eliminación de datos erróneos, filtrado de fines de semana y verificación de coherencia.
3. **Enriquecimiento temporal:** asignación del día de negociación y resamplado a intervalos de 4 horas.

4. **Cálculo de variables derivadas:** generación de datos acumulados y transformaciones básicas.
5. **Ingeniería de características:** construcción de *features* avanzadas y análisis para evitar *data leakage*.
6. **Selección de características:** filtrado manual y mediante modelos tradicionales con mayor explicabilidad.
7. **Definición y cálculo de los *targets*:** construcción de las variables objetivo para los modelos supervisados.
8. **Entrenamiento y evaluación:** ajuste de hiperparámetros, búsqueda de umbrales óptimos y evaluación con métricas de clasificación, regresión y métricas financieras.
9. **Preparación para despliegue:** guardado de modelos, *scalers*, configuración y lista final de *features*.
10. **Obtención de información actualizada de los futuros:** recopilación de márgenes iniciales y de mantenimiento, multiplicadores, *tick size*, *tick value*, horarios de negociación, spread estimado y slippage medio.
11. **Backtesting y simulación:** ejecución de backtests sobre datos históricos y simulaciones Monte Carlo para evaluar robustez y sensibilidad a escenarios adversos.
12. **Operativa y monitorización:** ejecución del sistema, optimización de la cartera, control de riesgos, monitorización continua y análisis de rendimiento.
13. **KPI:** dockerización del sistema, integración con Prometheus, Loki y SQL para métricas y logs, visualización mediante grafana
14. **Alertas:** Vigilancia de las métricas y logs que al superar umbrales definidos envían alertas a Slack a través de Alertmanager

El sistema operará en interdía ya que este marco temporal ofrece diferentes ventajas para sistemas de aprendizaje automático. En primer lugar, los datos presentan menor ruido microestructural, lo que facilita el aprendizaje de patrones relevantes y disminuye el riesgo de *overfitting*. Además, los modelos no sufren por la alta no estacionalidad y los cambios de volatilidad que existen en los datos intradía. A nivel operativo, el trading interdía requiere menor monitorización continua de la situación, dado que se ejecutan menos operaciones y se mantiene las posiciones durante varios días o incluso semanas. Por otro lado, las estructuras de precio basadas en swings son más claras y consistentes en marcos interdía, lo que las hace especialmente adecuadas para la extracción de características y el análisis estructural del mercado.

3.2 Desarrollo del autómata

Uno de los objetivos principales del proyecto es garantizar que el sistema sea modular, reutilizable y fácilmente extensible, independientemente del activo financiero con el que se trabaje.

Para ello, se ha diseñado una arquitectura que separa claramente las responsabilidades y permite incorporar nuevos componentes sin modificar el núcleo del sistema.

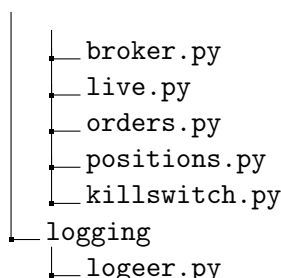
Con este propósito, la lógica del código se ha implementado dentro de un paquete instalable de Python, utilizando la instalación en modo editable mediante `pip install -e`. Esta decisión facilita el desarrollo iterativo, permite mantener una estructura coherente del proyecto y asegura que cualquier módulo pueda importarse desde otros scripts o notebooks sin necesidad de duplicar código.

La organización del paquete sigue una estructura modular que agrupa las funcionalidades por dominios lógicos, permitiendo añadir nuevos activos, modelos o pipelines de datos sin alterar la arquitectura existente.

```

/
├── paths.py
├── get_info
├── data
│   ├── loader.py
│   ├── cleaner.py
│   ├── resampler.py
│   └── to_parquet.py
├── features..... Gran cantidad de archivos para ordenar las funciones que generar las
│   │   características
├── supervised
│   ├── load.py
│   ├── models.py
│   ├── target.py
│   ├── pretrain.py
│   ├── train.py
│   ├── posttrain.py
│   ├── evaluate.py
│   └── backtest.py
├── portfolio
│   ├── optimizer.py
│   └── backtest.py
├── risk
│   ├── drawdown.py
│   └── spread.py
└── execution

```



El orden mostrado en la estructura anterior refleja la secuencia lógica en la que se invocan los distintos módulos del sistema. Cada directorio agrupa funcionalidades relacionadas y el flujo de ejecución avanza de forma coherente desde la carga y preparación de los datos, pasando por la generación de características y el entrenamiento de los modelos supervisados, hasta la optimización de cartera, la gestión del riesgo y la ejecución en tiempo real. Esta organización modular permite mantener un código limpio, fácilmente extensible y con responsabilidades claramente separadas.

Algunas funciones presentes en los distintos directorios del paquete son reutilizables en varias etapas del proceso, especialmente aquellas relacionadas con la obtención y carga de datos desde los archivos originales.

El código completo puede consultarse en el repositorio referenciado en el Anexo C.

A continuación se describen las distintas fases del flujo de datos y las funciones principales utilizadas en cada una de ellas.

3.2.1 Obtención de datos de los futuros

Se proporcionan los datos históricos OHLCV de distintos futuros con los que va a trabajar. Sin embargo, existen diferentes datos que no se han proporcionado, destacando el mercado donde cotiza cada futuro y la divisa con la que se negocia, estos dos datos son necesarios para posteriormente poder obtener el resto de información necesaria. Una vez identificados el mercado y la moneda de cada futuro, es posible conectarse a la API de Interactive Brokers y realizar peticiones para obtener información adicional. Esta información puede clasificarse en dos grandes grupos: **información estática** e **información dinámica**.

Dentro de la información estática o casi estática ya que no rara vez cambia, además de lo mencionado anteriormente se encuentra el identificador único de cada futuro, el multiplicador del futuro (apalancamiento implícito), el tamaño mínimo de movimiento (*tick size*) y el valor monetario asociado a este movimiento (*tick value*).

Por otro lado, la información dinámica cambia cada día o semana. Entre estos datos se encuentran los horarios de negociación, que dependen tanto de la bolsa como de los festivos del país correspondiente. Esta información permite determinar si un contrato puede operarse en

un momento dado.

También se dispone de información sobre las horas de mayor liquidez, aunque en este proyecto se emplearán estadísticas derivadas de los propios datos para identificar los periodos más líquidos y así reducir el *slippage*, entendido como la diferencia entre el precio esperado al enviar una orden y el precio real al que se ejecuta.

Finalmente, dentro de la información dinámica destacan dos parámetros fundamentales: el *margen inicial* y el *margen de mantenimiento*. El margen inicial es la cantidad mínima necesaria para abrir una posición, mientras que el margen de mantenimiento es el capital requerido para mantenerla abierta. En operativa interdi a, este margen suele ser mayor para proteger al broker frente a posibles saltos bruscos de precio al inicio de la sesi n, conocidos como *Price Gap* u *Opening Gap*.

```
{
  "AD": {
    "name": "Australian Dollar",
    "symbol": "AD",
    "ib_symbol": "AUD",
    "exchange": "CME",
    "currency": "USD",
    "approx_total_fee_per_side": 2.5,
    "min_slippage_ticks": 1,
    "max_spread_ticks": 3,
    "fallback_margin": 3000
  }
}

{
  "AD": {
    "name": "Australian dollar",
    "ib_symbol": "AUD",
    "conId": 854527687,
    "exchange": "CME",
    "multiplier": 100000.0,
    "tick_size": 5e-05,
    "tick_value": 5.0,
    "currency": "USD",
    "trading_hours": "20260620:CLOSED; ...",
    "liquid_hours": "20260620:CLOSED; ...",
    "approx_total_fee_per_side": 2.5,
    "min_slippage_ticks": 1,
    "max_spread_ticks": 3,
    "min_profit_ticks": 3.0,
    "initial_margin": 3000,
    "maint_margin": 2700.0,
    "fetched_at": "2026-06-20T14:42:04.151746+00:00"
  }
}
```

Figura 3.1: Informaci n est tica y din mica del contrato de futuros AD obtenida mediante la API de Interactive Brokers.

3.2.2 Carga y limpieza

Se han facilitado los datos hist ricos OHLCV de distintos futuros con los que van a trabajar.

Tabla 3.1: Ejemplo de registro histórico OHLCV de un contrato de futuros

Ticker	Per	Fecha	Hora	Open	High	Low	Close	Volumen	OpenInt
AD	I	20010502	060100	0.52000	0.52000	0.52000	0.52000	2	0

Como se puede observar en la Tabla 3.1 la información facilitada incluye datos sobre el activo representado, la fecha y la hora de registro, los datos OHLCV y el interés inicial de cada día, este último dato no será utilizado para simplificar la complejidad y porque al ser un valor no relativo o porcentual no es tan recomendado para los modelos de aprendizaje profundo, además de que el interés puede cambiar a lo largo del día.

En un primer análisis se observa que las marcas temporales de los datos no están estandarizadas al formato UTC, por lo que este es el primer aspecto que debe corregirse. Para identificar la zona horaria original de cada conjunto de datos, se utilizan como referencia los horarios de apertura y cierre de cada futuro. Agrupando el volumen por horas y analizando los periodos con actividad reducida o nula, es posible detectar de forma fiable las horas en las que el mercado permanece cerrado y, a partir de ello, inferir la zona horaria utilizada en los datos históricos.

La Figura 3.2 muestra un ejemplo del procedimiento utilizado para identificar la zona horaria original de los datos. En el caso del futuro del oro (GC), puede observarse que existe una pausa diaria en el volumen entre las 23:00 h y las 24:00 h, correspondiente al cierre habitual del mercado CME Globex. Esta ventana de inactividad permite inferir que las marcas temporales del dataset están expresadas en horario de Chicago, lo que facilita su conversión posterior a UTC.

Este proceso fue uno de los pocos que no pudo automatizarse completamente, ya que requiere interpretar patrones específicos de cada mercado. No obstante, el comportamiento es suficientemente consistente como para generalizarlo: los futuros negociados en el CME utilizan habitualmente la zona horaria de Chicago (UTC-6), mientras que los futuros europeos suelen emplear la zona horaria de Europa Central (UTC+1). A partir de esta información es posible estandarizar todos los datos al formato UTC de manera fiable.

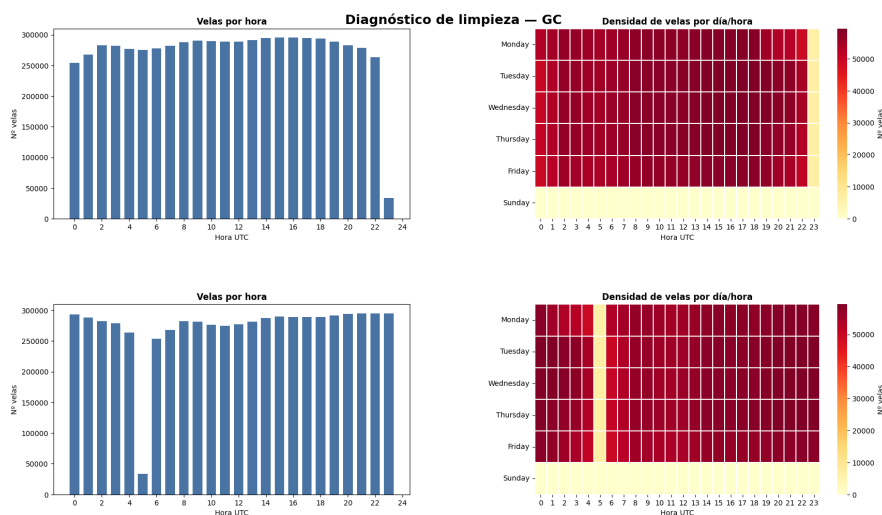


Figura 3.2: Comparación antes y después de ajustar

Después del análisis preliminar, el primer paso consiste en eliminar aquellos registros que claramente no deberían aparecer en la serie temporal, en particular los correspondientes a fines de semana, ya que los futuros no cotizan durante esos periodos y cualquier dato presente suele deberse a errores del proveedor o desajustes en los timestamps.

Los datos en horas de descanso si representan actividad real aunque sea mínima, pueden ser órdenes que llegan tarde, que se cruzan en el libro de órdenes aunque el mercado esté cerrado, actividad residual del Globex, ajustes del Exchange, ticks de apertura o cierre del libro. Además eliminarlos elimina la continuidad temporal, introduce gaps artificiales y puede afectar a diferentes métricas.

Una vez estandarizadas las marcas temporales, se incorpora una primera columna derivada que identifica el día de trading. Aunque esta variable no se utilizará directamente como entrada para los modelos, resulta útil para calcular agregados diarios que posteriormente servirán para construir otras características más avanzadas.

Con el objetivo de mejorar el rendimiento y reducir el tamaño de los archivos, los datos originalmente almacenados en formato CSV se convierten a formato Parquet. Además de ofrecer una compresión más eficiente y una lectura significativamente más rápida, Parquet permite organizar los datos en particiones independientes. Esto facilita cargar únicamente los segmentos necesarios, por ejemplo, por año o por mees, optimizando tanto el tiempo de acceso como el uso de memoria durante las fases de entrenamiento y análisis.

3.2.3 Cálculo de características

Una vez con los datos OHLCV limpios, se procede a generar el conjunto de características derivadas que servirán como entrada para los modelos de aprendizaje automático. Para ello se emplean funciones procedentes de distintas librerías.

Las características generadas abarcan múltiples categorías: de momento, estructurales, temporales, de tendencia, de volatilidad, de volumen y otras derivadas de combinaciones entre ellas. Dado que la estrategia es de tipo multidía, se incorporan además *swing charts* y un conjunto de variables construidas a partir de su estructura

Antes de calcular estas variables, los datos originales, registrados con una resolución de un minuto, se agregan en velas de 4 horas. Esta frecuencia reduce el ruido propio de las señales de muy corto plazo y resulta adecuada para una estrategia interdía, en la que las posiciones pueden permanecer abiertas durante varios días.

Para mantener una organización clara y facilitar la extensibilidad del sistema, las características se agrupan en subcarpetas según su tipología. En aquellos casos donde existe un número elevado de funciones, se realiza una subdivisión adicional para mantener una estructura modular y fácilmente mantenible.

derived

Características derivadas de otras series básicas. Incluye transformaciones estadísticas, normalizaciones, estandarizaciones y combinaciones no triviales de indicadores primarios.

momentum

Indicadores de momento y aceleración del precio. Contiene osciladores, ROC (Rate of Change) y medidas de fuerza relativa basadas en variaciones porcentuales.

returns

Cálculo de retornos simples, logarítmicos y acumulados. Incluye funciones para medir variaciones relativas entre barras y construir series de rendimiento.

selection

Módulo destinado a la selección de características. Puede incluir filtros, ranking de importancia, eliminación de colinealidad o criterios de calidad.

structural

Características que describen la estructura interna de las velas y del microcomportamiento del precio. Incluye patrones de velas, proporciones OHLC y medidas de microestructura.

swings

Detección de swings, pivotes y estructuras de máximos/mínimos relevantes. Útil para identificar cambios de tendencia y puntos de giro.

temporal

Variables temporales: hora, minuto, día de la semana, estacionalidades, ciclos intradía y cualquier codificación temporal relevante.

trend

Indicadores de tendencia: medias móviles, MACD, ADX, fuerza de tendencia y medidas de persistencia direccional.

volatility

Medidas de volatilidad histórica y realizada. Incluye ATR, bandas de Bollinger, rangos, dispersión de precios y estimadores de volatilidad intradía.

volume

Características basadas en volumen: flujo de órdenes, volumen relativo, volumen por precio, desequilibrio comprador/vendedor y estadísticas derivadas del volumen.

Una vez generadas todas las características, se realiza una primera validación para identificar la presencia de valores nulos en cada una de ellas. En este análisis destacan especialmente los swings, que por su propia naturaleza presentan numerosos huecos: solo se actualizan cuando aparece un nuevo máximo o mínimo significativo. En estos casos se opta por aplicar un forward fill, propagando el último valor válido disponible, ya que esta aproximación mantiene la coherencia estructural del indicador sin introducir información futura.

Algunas características requieren un número mínimo de velas para poder calcularse, por ejemplo, indicadores que necesitan 12 o 14 observaciones iniciales, lo que genera un pequeño bloque de filas incompletas al comienzo de la serie. Estas primeras filas se eliminan sin impacto relevante, ya que representan una fracción insignificante del total de datos disponibles. Debido a esta dependencia de ventanas largas, no resulta viable reiniciar los indicadores al comienzo de cada día, una idea inicialmente considerada. Además, dado que la estrategia es multidía, reiniciar los cálculos diarios tampoco tendría sentido operativo, pues rompería la continuidad temporal necesaria para capturar movimientos prolongados.

En una primera versión del proyecto se incluyeron únicamente las características propuestas en [2]. Sin embargo, con el objetivo de enriquecer la representación del mercado y mejorar la capacidad predictiva de los modelos, se decidió ampliar el conjunto incorporando características adicionales de mayor complejidad, incluyendo indicadores estructurales, relaciones entre activos y métricas derivadas de swing charts.

3.2.4 Selección de características

Una vez calculadas todas las características, es necesario reducir su número, ya que muchas de ellas pueden estar altamente correlacionadas o aportar información redundante. Para ello se aplican varias técnicas complementarias.

El primer paso consiste en eliminar las características con alta correlación entre sí, evitando así que el modelo reciba señales duplicadas que puedan inducir sobreajuste. También se descartan aquellas variables que, por su definición, podrían introducir data leakage, así como las características no cíclicas o no estacionarias que podrían confundir al modelo, por ejemplo, valores que aumentan con el tiempo y que podrían ser interpretados erróneamente como señales predictivas.

Posteriormente se aplica un filtrado mediante el algoritmo LASSO, utilizando un modelo clásico de scikit-learn que ofrece una mayor interpretabilidad. Este método permite identificar las características con mayor peso explicativo y seleccionar aproximadamente el 50 % más relevante, reduciendo de forma significativa la dimensionalidad del conjunto de datos.

En conjunto, estos pasos ayudan a mitigar la maldición de la dimensionalidad. Aunque este fenómeno afecta especialmente a los modelos clásicos de aprendizaje automático, también resulta beneficioso en arquitecturas más complejas, ya que reduce el riesgo de sobreajuste y disminuye los tiempos de entrenamiento.

3.2.5 Definición y cálculo de la clase objetivo

Para este proyecto se ha decidido usar el método de la *triple barrera*. Este enfoque define tres posibles eventos que pueden ocurrir tras la apertura de una operación: la activación del *Stop Loss* (SL), la activación del *Take Profit* (TP) o el vencimiento de una barrera temporal. A partir de un horizonte fijo, se determina cuál de las tres barreras es alcanzada primero. Si el precio toca el nivel de pérdida predefinido, se clasifica como SL; si alcanza el nivel de beneficio, como TP; y si no se produce ninguno de los dos movimientos dentro del tiempo establecido, se asigna la clase correspondiente a la barrera temporal. En el caso de que en una misma vela se alcancen simultáneamente SL y TP, se adopta un criterio conservador asignando el peor de los dos escenarios, es decir, SL. Los umbrales de SL y TP se definen como múltiplos del ATR, lo que permite adaptarlos a la volatilidad del activo.

Durante la fase inicial del proyecto se consideró la posibilidad de predecir directamente patrones de *swings*. Sin embargo, estos presentan varias limitaciones, entre ellas la necesidad de velas de confirmación (habitualmente dos), lo que introduce un retraso operativo significativo, en este caso, aproximadamente ocho horas. Por este motivo se descartó su uso como objetivo de predicción, aunque sí se mantienen como características relevantes para el entrenamiento de los modelos.

Además de la etiqueta discreta derivada de la triple barrera, se calcula también el retorno asociado al movimiento, concretamente el retorno logarítmico. Esta información resulta útil para la capa de regresión descrita en la siguiente sección y permite evaluar con mayor precisión la calidad de las señales: no es equivalente un acierto con un retorno del 0.2 % que uno del 2 %. El retorno logarítmico se emplea por sus propiedades matemáticas: aditividad tempo-

ral, simetría, aproximación a la normalidad y comparabilidad entre activos con volatilidades distintas.

En términos generales, el espacio de posibles objetivos es amplio: etiquetas binarias (sube/baja), ternarias (SL/TP/tiempo), regresión de retornos, clasificación de magnitud del movimiento, predicción de swings o incluso estimación de probabilidad de tocar cada barrera. Cada uno de estos enfoques implica distintos requisitos de modelado y diferentes riesgos de *data leakage*.

3.2.6 Aprendizaje supervisado

Llegados al núcleo del proyecto, el siguiente paso consiste en definir las arquitecturas de los modelos de aprendizaje supervisado que se van a emplear. La arquitectura se resume en un sistema de dos modelos paralelos con tres capas secuenciales y acoplados mediante un mecanismo de soft-voting que combina las predicciones de cada etapa de clasificación.

Las dos primeras etapas son de clasificación binaria y la tercera de regresión. Ambos modelos están definidos usando PyTorch. Cada uno de los modelos seleccionados tiene una función, LSTM está orientado a capturar dependencias a largo plazo y patrones más suaves en la secuencia, mientras que GRU tiende a reaccionar mejor a cambios rápidos, transiciones de régimen y estructuras más volátiles.

En una primera versión con solo las dos fases de clasificación se opta por realizar una votación equitativa, y si la media supera un umbral indicado para cada fase se avanza a la siguiente. Posteriormente se incorporó la tercera capa de regresión para dinamizar la votación, usando el retorno esperado como ponderación. De este modo, si uno de los modelos estima un retorno significativamente mayor, su predicción adquiere más peso en la decisión final. El retorno se calcula a un horizonte de 12 velas.

Inicialmente se consideró complementar estos modelos con algoritmos clásicos como XGBoost o Random Forest, capaces de capturar relaciones no temporales. Sin embargo, se decidió centrar el estudio en arquitecturas profundas recurrentes, más adecuadas para

3.2.6.1 Arquitectura de los modelos

Los tres modelos LSTM comparten la misma estructura general:

- **Capa recurrente:** LSTM con 2 capas, 128 unidades ocultas, `batch_first=True` y `dropout=0.3`.
- **Extracción del estado:** se utiliza el último *hidden state* h_t de la última capa.
- **Cabeza densa:**

- Lineal(128 $\rightarrow$ 64), GELU, Dropout(0.3)
- Lineal(64 $\rightarrow$ 32), GELU, Dropout(0.2)
- Lineal(32 $\rightarrow$ salida)

Las salidas dependen del nivel:

- **L1:** Clasificación binaria (HOLD / ACCIÓN)
- **L2:** Clasificación binaria (SELL / BUY)
- **L3:** Regresión (retorno logarítmico)

Los modelos GRU son análogos a los LSTM, sustituyendo la capa recurrente por una GRU con la misma configuración (2 capas, 128 unidades, dropout).

3.2.6.2 Motivación de las dos fases de clasificación

La decisión de utilizar dos etapas de clasificación se debe al fuerte desbalanceo presente en el objetivo generado mediante la triple barrera. Aproximadamente el 60 % de las velas terminan tocando la barrera del tiempo y mientras que las otras dos barreras solo representan un 20 % cada una. En un inicio se probaron modelos con salida ternaria, pero incluso ajustando los pesos de las clases, los resultados no fueron satisfactorios.

Por ello se optó por dividir el problema en dos fases binarias:

1. **L1:** ¿Existe movimiento significativo? (¿toca SL o TP?)
2. **L2:** En caso afirmativo, ¿en qué dirección ocurre el movimiento?

En ambas fases se evalúa la probabilidad predicha y se compara con un umbral. Aunque inicialmente se utilizaron valores fijos, finalmente ambos umbrales se seleccionaron mediante un proceso de grid search diseñado para maximizar el *F1-score* en cada nivel. También se evaluó la posibilidad de decidir la clase únicamente por diferencia relativa entre probabilidades, pero el uso de un umbral explícito, optimizado para cada fase, proporcionó una mayor estabilidad y un mejor equilibrio entre precisión y recall.

Antes de pasar al entrenamiento es necesario calcular los hiperparámetros idóneos para cada entrenamiento teniendo en cuenta el tamaño de cada dataset, el periodo de muestreo y el número de velas a futuro a predecir. Se obtienen así los siguiente hiperparámetros.

lookback

Número de velas hacia atrás que se pasa a cada momento temporal

train_size

Tamaño del bloque de entrenamiento para cada iteración

test_size

Tamaño del bloque de prueba para cada iteración

gap

Número de velas de separación entre el final de una iteración y el comienzo de la siguiente.
Sirve para evitar filtrado de datos

3.2.6.3 Entrenamiento de los modelos

El entrenamiento de los modelos se realiza de forma independiente para cada arquitectura por separado, entrenando las tres fases juntas. Para ello, se emplea un sistema de *walk forward* con *moving window*, es la técnica estándar para entrenamiento con series temporales. A diferencia del entrenamiento normal este método respeta la estructura temporal de los datos y evita data leakage.

El método *walk forward* (también conocido como *rolling forecast origin*) consiste en entrenar el modelo en un bloque de inicial de datos históricos y evaluarlo en el siguiente bloque inmediatamente posterior. A continuación se avanza hacia el siguiente bloque dejando algunas velas de margen y se repite este proceso hasta completar el entrenamiento con todos los datos posibles. La variante utilizada, *moving window*, mantiene un tamaño fijo de ventana en cada iteración, a diferencia de la opción *expanding window*, donde la ventana crece progresivamente.

Este enfoque es imprescindible porque el *cross-validation* clásico no es válido en series temporales: al mezclar aleatoriamente los datos y dividirlos en *folds* sin respetar el orden temporal, permite que el modelo entrene con información futura, introduciendo un *data leakage* masivo.

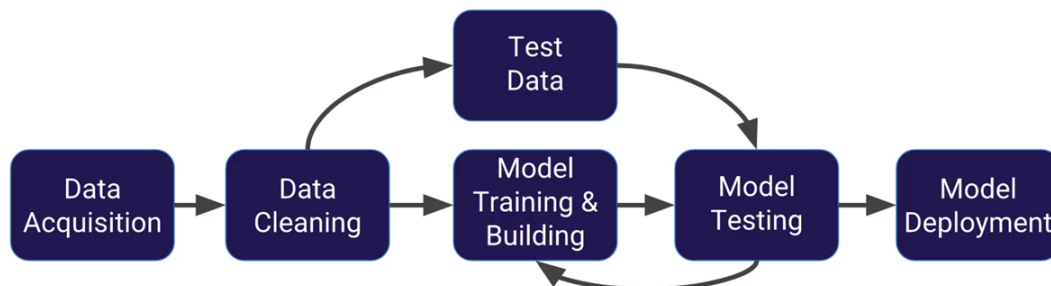


Figura 3.3: Añadir img de walk forward

No olvidar

Una vez definidos los hiperparámetros, se generan los datasets correspondientes a cada iteración del walk forward, incorporando el *lookback* calculado previamente. Para cada nivel del modelo se separan los datos en entrenamiento y prueba, y se aplica un *scaler* ajustado exclusivamente con los datos de entrenamiento, evitando así cualquier fuga de información. El orden de entrenamiento es secuencial: primero L1, después L2 y finalmente L3. El resultado de cada iteración es un conjunto de predicciones y probabilidades asociadas a cada nivel.

Tras completar el entrenamiento, se procede a seleccionar los umbrales óptimos para las dos primeras capas de clasificación. Dado que la tercera capa es de regresión, no dispone de probabilidad asociada. Los umbrales se determinan mediante un proceso de *grid search* orientado a maximizar el *F1-score macro*, métrica adecuada para escenarios con fuerte desbalanceo de clases, a diferencia de *accuracy*, *precision* o *recall*, que pueden resultar engañosas en este contexto.

Los modelos entrenados se guardan en archivos `.pth`, junto con el último *scaler* utilizado en la ventana correspondiente. Esto es necesario porque el *scaler* se recalcula en cada iteración y sus parámetros (mediana, IQR, etc.) cambian con el tiempo. También se almacenan las características empleadas, para poder reproducir el filtrado durante el backtest y la operativa real, así como los parámetros del modelo, incluyendo los umbrales seleccionados.

El *scaler* elegido es **RobustScaler**, debido a que utiliza la mediana y el rango intercuartil (IQR), lo que lo hace resistente a valores atípicos y a movimientos extremos del mercado, conocidos como *cisnes negros*. Alternativas como *StandardScaler* o *MinMaxScaler* presentan desventajas: el primero es sensible a outliers y el segundo fija un rango que puede volverse inestable ante cambios de régimen.

La votación entre modelos solo se realiza si ambos superan la primera fase y coinciden en la dirección predicha en la segunda. En caso contrario, no se genera señal. La ponderación final incorpora el retorno estimado por la capa de regresión, de modo que predicciones con mayor retorno esperado tienen mayor peso.

Una posible extensión futura consiste en actualizar tanto el modelo como el *scaler* de manera periódica (por ejemplo, semanalmente), permitiendo que el sistema se adapte mejor a cambios de régimen del mercado. Esta actualización no solo implicaría volver a entrenar desde cero, sino aplicar un proceso de *fine tuning* sobre los modelos ya existentes. El *fine tuning* permite ajustar los pesos previamente aprendidos utilizando únicamente los datos más recientes, reduciendo el coste computacional y acelerando la adaptación del modelo a nuevas condiciones de volatilidad, liquidez o comportamiento direccional. De este modo, el sistema podría mantener la estabilidad de los patrones aprendidos a largo plazo mientras incorpora información fresca del mercado, logrando un equilibrio entre memoria histórica y capacidad de reacción.

3.2.6.4 Análisis del entrenamiento

Una vez finalizado el entrenamiento de los modelos, es necesario evaluar su rendimiento utilizando métricas tanto de clasificación como de regresión. El objetivo es comprobar no solo la capacidad predictiva de cada nivel, sino también la estabilidad del entrenamiento y la coherencia de las predicciones.

En el caso de las dos primeras capas, correspondientes a problemas de clasificación binaria, se emplean métricas estándar como el *classification report*, que incluye precisión, recall y F1-score por clase. Además, se calculan métricas adicionales que permiten evaluar distintos aspectos del comportamiento del modelo:

- **AUC-ROC:** mide la capacidad del modelo para separar ambas clases en distintos umbrales.
- **Curva de fiabilidad (reliability curve):** evalúa el grado de calibración de las probabilidades predichas.
- **Matriz de confusión con conteo:** permite analizar los aciertos y errores por clase.
- **Recall y F1-score por clase:** especialmente relevantes en escenarios con fuerte desbalanceo.
- **Pérdida de entrenamiento y prueba por fold:** se registra la pérdida de la última época para cada ventana del *walk forward*, lo que permite detectar sobreajuste o inestabilidad.

Para la tercera capa, dedicada a la regresión del retorno logarítmico, se emplea un conjunto de visualizaciones y métricas que permiten evaluar la calidad de las predicciones:

- **Gráfico predicción vs. valor real (scatter plot):** muestra la relación entre el retorno predicho y el observado.
- **Distribución del error:** permite analizar la simetría y dispersión de los residuos.
- **Densidad conjunta:** evalúa la concentración de predicciones en distintas zonas del espacio real-predicho.
- **Residual vs. fitted:** ayuda a detectar patrones no capturados por el modelo o heterocedasticidad.

En conjunto, estas métricas permiten obtener una visión completa del rendimiento del sistema: desde la capacidad de clasificación en las dos primeras fases hasta la calidad de la estimación del

retorno en la tercera. Además, el análisis por *folds* del *walk forward* proporciona información sobre la estabilidad temporal del modelo y su capacidad para generalizar en distintos periodos del histórico.

3.2.7 Backtest de las predicciones

Una vez verificados los resultados del entrenamiento, se procede a realizar el *backtest* con el objetivo de evaluar el comportamiento del sistema en condiciones lo más cercanas posible a la operativa real. Aunque esta simulación no incorpora ciertos elementos propios del mercado, como la latencia, constituye un punto de partida sólido para analizar la robustez de las señales generadas por los modelos.

El backtest se ejecuta utilizando datos con resolución de 4 horas debido a limitaciones de la API empleada para la descarga. Sobre estos datos se prueban cuatro estrategias operativas:

3.2.7.1 Fase 1: Selección de predicciones válidas y cálculo de PnL teórico

En esta fase se generan las predicciones finales y se calcula el PnL teórico de cada una. A partir de esta información se definen cuatro criterios alternativos para decidir qué predicciones se consideran válidas:

	Long y Short	Solo Long
Todas las predicciones	Caso 1 Se aceptan todas las predicciones que superan los umbrales de L1 y L2	Caso 2 Igual que el Caso 1, pero solo se permiten operaciones long
Predicciones que superan un umbral más estricto	Caso 3 Solo se aceptan predicciones con retorno esperado $ L3 $ superior al percentil 70 (quantile 0.7)	Caso 4 Igual que el Caso 3, pero solo se permiten operaciones long

Tabla 3.2: Criterios de selección de predicciones válidas en la Fase 1 del backtest.

El percentil 70 se calcula como:

$$\text{threshold}_{L3} = |L3|_{0,7}$$

Con estos cuatro criterios se calcula el Sharpe Ratio teórico de cada conjunto de predicciones antes de pasar a la fase operativa.

3.2.7.2 Fase 2: Backtest operativo con cuatro modalidades

Una vez seleccionadas las predicciones válidas, se ejecuta el backtest operativo. En esta fase sí se simula la apertura y cierre de posiciones, aplicando cuatro modalidades distintas:

	Scale-in: No	Scale-in: Sí
Sizing dinámico: No	Estrategia 1 1 posición máxima tamaño fijo (1)	Estrategia 2 hasta 5 posiciones tamaño fijo (1)
Sizing dinámico: Sí	Estrategia 3 1 posición máxima tamaño variable (≤ 5)	Estrategia 4 hasta 25 posiciones tamaño variable (≤ 5)

Tabla 3.3: Modalidades operativas utilizadas en la Fase 2 del backtest.

3.2.7.3 Métricas de evaluación

Para evaluar el rendimiento se utilizan las siguientes métricas:

Max Drawdown

El *Max Drawdown* mide la mayor caída relativa del capital desde un máximo histórico hasta un mínimo posterior. Es una métrica fundamental para evaluar el riesgo, ya que cuantifica la peor racha de pérdidas acumuladas que puede experimentar la estrategia. Un drawdown elevado indica periodos prolongados de pérdidas y menor estabilidad operativa.

$$\text{MDD} = \max_t \left(\frac{\max_{s \leq t} \text{Equity}(s) - \text{Equity}(t)}{\max_{s \leq t} \text{Equity}(s)} \right)$$

Sharpe Ratio

El *Sharpe Ratio* mide el retorno ajustado por riesgo, comparando el rendimiento medio del sistema con la volatilidad de sus retornos. Un Sharpe alto indica que la estrategia genera beneficios consistentes con una variabilidad relativamente baja. En este proyecto, el Sharpe se calcula utilizando como capital inicial un valor equivalente a **10 veces el *initial margin*** del activo, lo que permite normalizar la métrica entre activos con diferentes requisitos de margen.

$$\text{Sharpe} = \frac{E[R_p]}{\sigma_p}$$

Calmar Ratio

El *Calmar Ratio* relaciona el retorno anualizado con el riesgo máximo asumido (Max

Drawdown). A diferencia del Sharpe, que se basa en la volatilidad, el Calmar penaliza directamente las caídas profundas del capital. Es especialmente útil en estrategias con drawdowns prolongados o distribuciones de retornos no gaussianas.

$$\text{Calmar} = \frac{\text{Retorno anualizado}}{\text{Max Drawdown}}$$

CAGR (Compound Annual Growth Rate)

El *CAGR* representa la tasa de crecimiento anualizada del capital, suponiendo que los beneficios se reinvierten y que el crecimiento es compuesto. Es una métrica clave para comparar estrategias con diferentes duraciones, ya que resume el rendimiento total en una única tasa anual equivalente. A diferencia del retorno total, el CAGR suaviza la variabilidad temporal y ofrece una visión más estable del crecimiento a largo plazo.

$$\text{CAGR} = \left(\frac{\text{Equity}_{\text{final}}}{\text{Equity}_{\text{inicial}}} \right)^{\frac{1}{\text{años}}} - 1$$

Opcionalmente, se puede incluir un contador de rachas de victorias y derrotas para analizar la estabilidad operativa del sistema.

3.2.8 Simulación de Monte Carlo

3.2.9 Simulación de Monte Carlo

Como paso posterior al backtest se realiza una simulación de Monte Carlo con el objetivo de estimar la dispersión de las métricas de rendimiento y evaluar la robustez del sistema frente a diferentes secuencias posibles de resultados. Existen dos enfoques principales para aplicar Monte Carlo en este contexto. El primero consiste en reordenar o remuestrear las predicciones ya generadas por los modelos durante el backtest. El segundo, más complejo, implica generar nuevos datos OHLCV sintéticos a partir de estadísticas como medias, varianzas o correlaciones, y volver a alimentar los modelos para obtener predicciones completamente nuevas.

En este proyecto se opta por la primera aproximación, ya que permite obtener una estimación fiable de la variabilidad de las métricas sin necesidad de generar datos sintéticos ni volver a entrenar los modelos. Además, este método es computacionalmente más eficiente y evita introducir supuestos adicionales sobre la dinámica de precios.

La técnica utilizada es un *overlapping block bootstrap*. A diferencia del bootstrap clásico, que remuestrea observaciones individuales, el block bootstrap remuestrea bloques completos de la serie para preservar parte de la estructura temporal, como autocorrelaciones de corto plazo o patrones locales. El uso de solapamiento garantiza que todos los segmentos posibles de la serie

original puedan aparecer en las simulaciones, evitando limitarse únicamente a bloques fijos y disjuntos. En este caso la idea inicial era la de emplear bloques de 24 velas, lo que equivale aproximadamente a una semana de predicciones, pero al contar con solo 5 meses de datos para el backtest y teniendo filtros bastantes restrictivos se optó al final por agrupar por días y así pasar en muchos casos de solo tener dos semanas a tener varios días con movimientos, haciendo más efectivo la simulación.

3.2.10 Gestión de cartera

Una vez entrenados los modelos para cada futuro, es necesario coordinar sus señales para evitar problemas de liquidez, sobreexposición o conflictos entre activos. Para ello, en esta versión del proyecto se emplea un método de gestión de cartera sencillo y manual, cuyo objetivo es garantizar que las operaciones se ejecutan de forma coherente con las restricciones de capital y riesgo.

El procedimiento seguido es el siguiente:

1. Se generan las predicciones para todos los futuros disponibles en el instante temporal considerado.
2. Se filtran aquellas predicciones que superan los umbrales definidos en las capas L1 y L2.
3. Las predicciones válidas se ordenan por nivel de confianza total, de mayor a menor.
4. Se recorre esta lista ordenada y, para cada predicción, se comprueba si cumple las restricciones operativas: disponibilidad de capital, límites de exposición por activo y restricciones de riesgo.
5. Si la predicción cumple las condiciones, se ejecuta la operación correspondiente; en caso contrario, se descarta y se pasa a la siguiente.

Este enfoque permite controlar la asignación de capital de manera simple y transparente, evitando situaciones en las que múltiples activos compitan por recursos limitados o generen niveles de riesgo excesivos. Aunque se trata de un método básico, resulta adecuado para una primera versión del sistema y permite evaluar el comportamiento conjunto de los modelos en un entorno multiactivo.

En esta fase se realiza además un *backtest* específico para la cartera completa. A diferencia del backtest individual por activo, cuyo objetivo era comprobar el funcionamiento general de los modelos y la coherencia de las señales, este backtest es más restrictivo, ya que incorpora todas las limitaciones reales de operativa además de márgenes iniciales y de mantenimiento que ya se tenían en cuenta antes: capital inicial de 1 000 000 USD, valor notional por contrato

y para la cartera completa, porcentaje máximo de riesgo por operación y una exposición total máxima del 25 % del capital disponible.

Del mismo modo, se ejecuta una nueva simulación de Monte Carlo a nivel de cartera, utilizando los PnL diarios resultantes de este backtest más estricto. La metodología es la misma que en la simulación individual, pero aplicada ahora a la serie conjunta de beneficios de toda la cartera, lo que permite analizar la dispersión de las métricas bajo un escenario multiactivo y con restricciones reales de riesgo.

La optimización formal de la cartera mediante técnicas matemáticas más avanzadas (por ejemplo, programación lineal o cuadrática) se considera una línea de trabajo futura, ya que no se ha implementado en esta versión del proyecto.

3.2.11 Manejo del riesgo

Una vez entrenados los modelos y antes de pasar a la operativa, es necesario definir el nivel de riesgo que se está dispuesto a asumir. La métrica principal utilizada en este proyecto es el *Max Drawdown*. Este valor actúa como límite superior de pérdidas aceptables: si la curva de capital alcanza dicho umbral, se cierran todas las posiciones abiertas para evitar un deterioro mayor del capital.

Además del drawdown, se establece un porcentaje máximo del equity que puede estar simultáneamente expuesto en posiciones abiertas. Este *Exposure Limit* define el capital máximo que puede quedar comprometido en operaciones activas, independientemente del apalancamiento implícito de cada instrumento. De este modo se evita que un conjunto de señales coincidentes genere un nivel de riesgo agregado excesivo.

Otro aspecto relevante es la gestión operativa en torno a la apertura y cierre de las sesiones. Durante estos periodos el *spread* suele ampliarse y la liquidez puede ser menor, lo que incrementa el riesgo de *slippage*. Para mitigar este efecto, se incorporan reglas basadas en la hora y el volumen: en la apertura se retrasa la toma de decisiones hasta 15 minutos después o hasta que el volumen supere un umbral mínimo, mientras que en el cierre se adelanta la ejecución si el volumen comienza a disminuir. Estas reglas permiten operar únicamente en condiciones de mercado más estables.

Finalmente, se realizan comprobaciones de integridad de los datos antes de alimentar los modelos: verificar que los precios *Open* y *Close* se encuentran entre *Low* y *High*, que *Low* es menor o igual que *High*, que el volumen no es negativo y que no existen duplicados. Estas validaciones son esenciales para evitar errores silenciosos que puedan afectar tanto al entrenamiento como a la operativa.

3.2.12 Operación

Una vez completado el entrenamiento de los modelos, la configuración del sistema y la definición de las reglas de riesgo, el último paso consiste en integrar todos los componentes en un único módulo operativo capaz de conectarse a la API de Interactive Brokers y ejecutar las operaciones en tiempo real.

El flujo general de la operación es el siguiente:

1. Conexión a la API de Interactive Brokers
2. Lectura de los futuros sobre los que se van a operar y obtener los contratos específicos a operar
3. Descarga de los datos históricos necesarios para realizar las predicciones iniciales
4. Carga de la información de configuración del sistema
5. Carga de los modelos, los *scalers*, y la configuración individual de cada futuro
6. Ejecución periódica de predicciones cada 4 horas
7. Activación de un *watchdog* encargado del control de riesgo y del estado del sistema.

Dado que los futuros tienen fecha de vencimiento, el sistema obtiene diariamente la lista de contratos activos para cada símbolo y los ordena por volumen con el fin de seleccionar el contrato más líquido, minimizando así el *spread*. Si existen posiciones abiertas en un contrato que deja de ser el más líquido, el comportamiento depende de la configuración: o bien se cierran las posiciones inmediatamente, o bien se compara la diferencia de volumen con un umbral. Si la diferencia es pequeña, se mantiene la posición abierta aunque las predicciones futuras se realicen sobre el nuevo contrato principal; si la diferencia supera el umbral, se cierran las posiciones y se migra al contrato más líquido.

Es posible configurar el comportamiento antes una predicción HOLD tras una predicción LONG o SHORT, existen dos configuraciones. En el primer caso, se decide seguir con la posición abierta, mientras que en el segundo caso, se cierra la posición. Algo parecido ocurre con predicciones sucesivas opuestas, se puede decidir cerrar la posición o cerrar la posición y abrir una nueva posición opuesta a la anterior.

Toda esta lógica se controla mediante un archivo de configuración en formato JSON. Además, cada operación ejecutada se almacena en un archivo JSON que registra toda la información relevante de la operación.

```
{
  "id": 1248,
  "symbol": "GC",
  "status": "closed",
  "side": "long",
  "quantity": 1,
  "timestamp_open": "2024-03-12T10:40:00Z",
  "timestamp_close": "2024-03-13T14:00:00Z",
  "entry_price": 2145.3,
  "exit_price": 2162.7,
  "stop_loss": 2138.0,
  "take_profit": 2170.0,
  "pnl": 174.0,
  "pnl_pct": 0.81,
  "return": 0.0081,
  "max_favorable_excursion": 0.0124,
  "max_adverse_excursion": -0.0048,
  "model_signal": "BUY",
  "model_confidence_l1": 0.87,
  "model_confidence_l2": 0.72,
  "model_reg_value": 0.0045,
  "thresholds_used": {
    "threshold_move": 0.55,
    "threshold_dir": 0.60
  },
  "commission": 2.5,
  "slippage": 0.0,
  "equity_before": 10234.5,
  "equity_after": 10408.5,
  "bar_index_open": 58291,
  "bar_index_close": 58347,
  "durationBars": 56,
  "duration_minutes": 1344
}
```

Figura 3.4: Ejemplo de registro JSON generado por el sistema para cada operación ejecutada.

3.2.13 Logging

El sistema incorpora un módulo de *logging* que actúa de forma transversal en todas las fases del flujo. Aunque no constituye una etapa independiente, es un componente esencial para garantizar la trazabilidad, la depuración y la supervisión del sistema.

Durante el entrenamiento se registran mensajes informativos sobre los activos procesados, los resultados obtenidos y posibles incidencias en los datos. En las fases de backtesting y simulación se almacenan métricas, configuraciones utilizadas y cualquier anomalía detectada.

El punto más crítico para el *logging* es la fase de operación. En este contexto se registran, entre otros:

- el estado de la conexión con la API,
- el contrato seleccionado para cada futuro,
- la correcta descarga y carga de datos,
- las configuraciones activas,
- las predicciones generadas,
- las órdenes enviadas y su estado,
- el número de posiciones abiertas,
- las alertas del *watchdog* de riesgo.

Este sistema de registro permite auditar cualquier operación, reproducir escenarios pasados y detectar rápidamente fallos o comportamientos inesperados.

3.3 Despliegue usando contenedores de Docker

Una vez desarrollado el autómata y verificado su funcionamiento, se ha optado por empaquetar el proyecto dentro de un contenedor Docker con el objetivo de facilitar su despliegue en otros equipos y garantizar la reproducibilidad del entorno. El uso de contenedores permite aislar las dependencias, fijar versiones concretas de los paquetes utilizados y asegurar que el sistema se ejecuta de forma idéntica independientemente de la máquina anfitriona.

3.3.1 Configuración del contenedor principal

Dado el diseño modular del sistema y por motivos de simplicidad y velocidad, todo el flujo de operación se ejecuta dentro de un único contenedor. Este contenedor incluye el código desarrollado, las dependencias necesarias y la lógica de operación.

El proceso de construcción del contenedor es sencillo: se copian los módulos del proyecto al interior de la imagen y se instalan las versiones específicas de las librerías utilizadas. De este modo se obtiene un entorno estático y controlado, evitando incompatibilidades entre versiones o diferencias entre entornos locales.

El contenedor ejecuta directamente el módulo principal de operación, encargado de la conexión con la API de Interactive Brokers, la descarga de datos, la generación de predicciones y la ejecución de órdenes. Para ello únicamente es necesario incluir en la imagen el archivo `P50peration.py` y el paquete desarrollado, ya que el entrenamiento de los modelos se realiza desde el entorno local y no forma parte del flujo operativo en tiempo

3.3.2 Bases de datos

Al ejecutar el sistema dentro de un contenedor, resulta natural incorporar también una base de datos para almacenar de forma persistente las métricas generadas durante la operación, el historial de operaciones y otros datos relevantes. Aunque inicialmente se utilizaban archivos JSON para registrar las operaciones, el uso de una base de datos relacional permite una gestión más estructurada, consultas más eficientes y una integración más sencilla con herramientas de visualización.

En este proyecto se utiliza PostgreSQL como motor de base de datos, acompañado de PgAdmin para facilitar la inspección manual de los datos. Parte de la información también puede visualizarse desde Grafana, descrito en la siguiente sección.

Dado que se almacenan distintos tipos de información, métricas, señales, riesgo, operaciones, etc, se han definido varias tablas especializadas, cada una con su propia estructura. Estas tablas se detallan a continuación:

Tabla 3.4: Estructura de la tabla `metrics`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Marca temporal de la métrica
metric_name	TEXT	Nombre de la métrica (pnl, sharpe, latency, etc.)
value	DOUBLE PRECISION	Valor numérico de la métrica
tags	JSONB	Información adicional (símbolo, timeframe, etc.)

Tabla 3.5: Estructura de la tabla `risk_events`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Momento del evento
event_type	TEXT	Tipo de evento (drawdown, latency, stale_data...)

Continúa en la siguiente página

Campo	Tipo	Descripción
severity	TEXT	Nivel (info, warn, critical)
details	JSONB	Información adicional del evento

Tabla 3.6: Estructura de la tabla `signals`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Marca temporal de la señal
symbol	TEXT	Símbolo del futuro
contract	TEXT	Contrato específico
signal_value	DOUBLE PRECISION	Valor de la señal
model_version	TEXT	Versión del modelo
features	JSONB	Features usadas en la predicción
decision	TEXT	accepted / rejected
reason	TEXT	Motivo del rechazo (spread, volumen, riesgo...)
tags	JSONB	Información adicional

Tabla 3.7: Estructura de la tabla `trades`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
symbol	TEXT	Símbolo del futuro
contract	TEXT	Contrato específico
status	TEXT	Estado de la operación
side	TEXT	Dirección
quantity	INTEGER	Número de contratos
timestamp_open	TIMESTAMPTZ	Momento de apertura
timestamp_close	TIMESTAMPTZ	Momento de cierre
entry_price	DOUBLE PRECISION	Precio de entrada
exit_price	DOUBLE PRECISION	Precio de salida
stop_loss	DOUBLE PRECISION	Stop loss asignado
take_profit	DOUBLE PRECISION	Take profit asignado
pnl	DOUBLE PRECISION	Beneficio/pérdida absoluto
pnl_pct	DOUBLE PRECISION	Beneficio/pérdida porcentual
return	DOUBLE PRECISION	Retorno relativo
max_favorable_excursion	DOUBLE PRECISION	Máximo movimiento a favor

Continúa en la siguiente página

Campo	Tipo	Descripción
max_adverse_excursion	DOUBLE PRECISION	Máximo movimiento en contra
model_signal	TEXT	Señal generada por el modelo
model_confidence_11	DOUBLE PRECISION	Confianza del modelo (nivel 1)
model_confidence_12	DOUBLE PRECISION	Confianza del modelo (nivel 2)
model_reg_value	DOUBLE PRECISION	Valor regresivo del modelo
thresholds_used	JSONB	Umbrales usados en la decisión
commission	DOUBLE PRECISION	Comisión aplicada
slippage	DOUBLE PRECISION	Slippage registrado
equity_before	DOUBLE PRECISION	Equity antes de la operación
equity_after	DOUBLE PRECISION	Equity después de la operación
bar_index_open	INTEGER	Índice de barra de apertura
bar_index_close	INTEGER	Índice de barra de cierre
durationBars	INTEGER	Duración en barras
durationMinutes	INTEGER	Duración en minutos

3.3.3 Visualización

Una vez que todo el sistema está desplegado en contenedores, se incorpora un módulo de visualización que permite monitorizar en tiempo real y en diferido toda la información generada durante la operación. Para ello se utilizan varios contenedores adicionales: *Pushgateway*, *Prometheus*, *Loki*, *Promtail* y *Grafana*.

Prometheus y Pushgateway se encargan de la gestión y recolección de métricas, mientras que Promtail y Loki gestionan los logs del sistema. Pushgateway y Promtail actúan como recolectores, y Prometheus y Loki como motores de almacenamiento y consulta. Por su parte, Grafana se conecta tanto a Prometheus como a Loki y también directamente al contenedor de PostgreSQL, lo que permite visualizar métricas, logs y datos almacenados en la base de datos desde un único panel.

Toda la información se consume después en un *dashboard* en grafana donde se agrupa toda la información.

Toda esta información se integra en un *dashboard* de Grafana donde se agrupan las métricas más relevantes. Entre ellas se incluyen indicadores de rendimiento como la *equity curve*, el *max drawdown*, el *Sharpe ratio*, la volatilidad diaria, el *profit factor*, el *win rate* y el rendimiento por futuro. También se muestran métricas operativas, logs del sistema y estadísticas de riesgo.

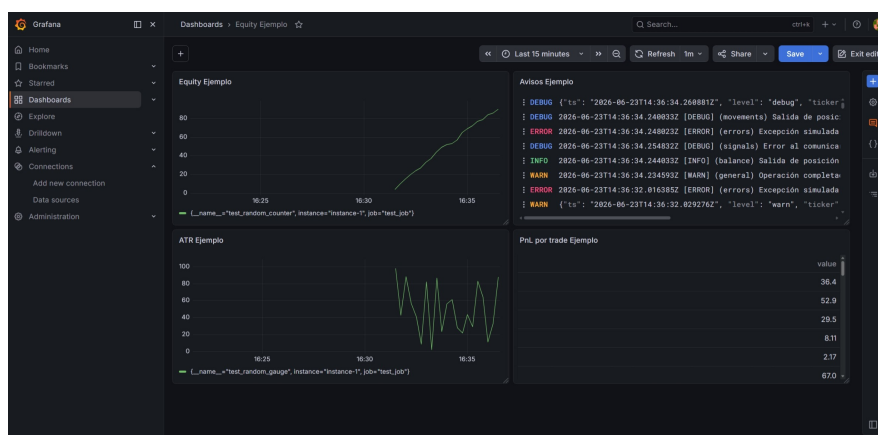


Figura 3.5: Ejemplo de un *Dashboard* en Grafana

3.3.4 Alertas y avisos

Además de la visualización en Grafana, se incorpora un sistema de alertas para recibir notificaciones ante eventos relevantes. Estas alertas se definen mediante reglas en *Alertmanager*, que forma parte del ecosistema de Prometheus. Cuando alguna de las reglas se activa, por ejemplo, un error en la conexión con la API, un aumento inesperado del riesgo, un fallo en la descarga de datos o un comportamiento anómalo del sistema, Alertmanager envía una notificación al dispositivo móvil del usuario.

En este proyecto las alertas se envían a través de Slack, utilizando una API key configurada previamente. Esto permite recibir avisos en tiempo real incluso cuando el sistema está ejecutándose de forma desatendida.

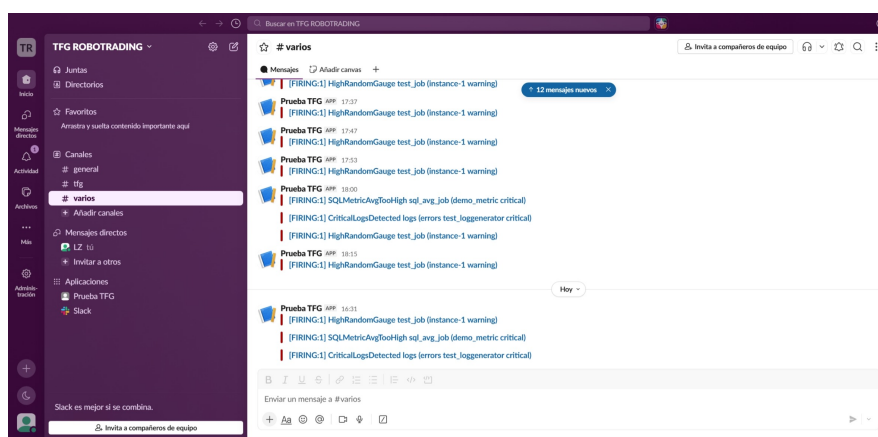


Figura 3.6: Ejemplo con alertas recibidas en Slack

Resultados

4.1 Resultados

Como se han mencionado en capítulos anteriores el resultado de las pruebas ha sido positivo

En la gráfica siguiente se muestra cómo va evolucionando el capital de la cuenta simulada, se puede observar todas las predicciones que realizan los modelos y el sharpe ratio final.

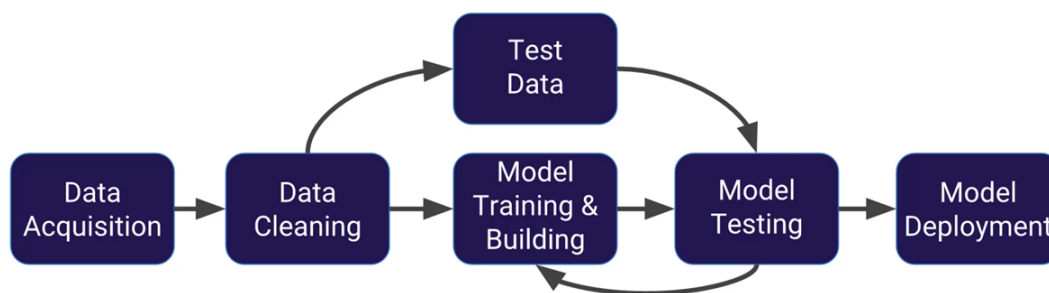


Figura 4.1: Añadir img con equity curve, max drawdown y sharpe ratio

Para poner en perspectiva los resultados he decidido compararlos con los obtenidos por los participantes de la competición Robotrader, competición en la cuál iba a participar en un inicio pero que por incompatibilidad con el desarrollo de este proyecto no acabé participando. Como se puede observar la curva mejora a muchos de los sistemas presentados.

La competición se realizó durante los meses de marzo a mayo, estos datos nunca se han usado para entrenar los modelos para evitar leakage. No es una comparación perfecta ya que no se tienen en cuenta todos los problemas que pueden haber en vivo pero se han intentado simular lo mejor posible.

Además de mostrar los datos teóricos que se hubiesen conseguido también he querido comparar con los datos de la simulación de Monte Carlo.

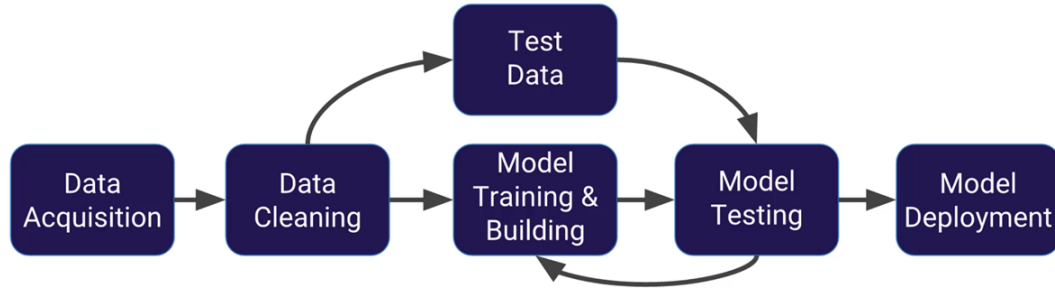


Figura 4.2: Añadir img comparativa del Backtest vs Robotrader

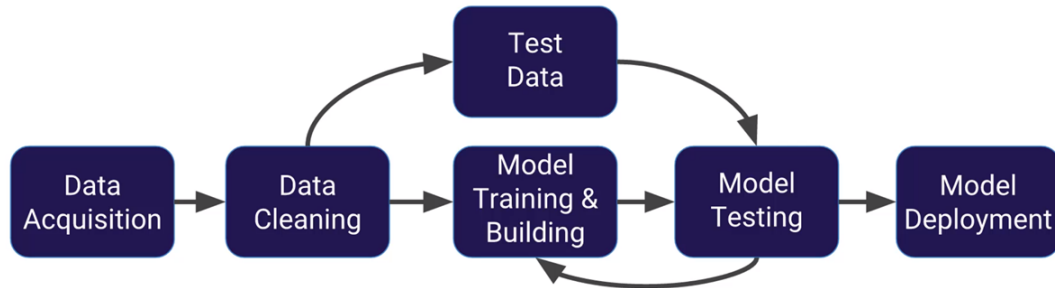
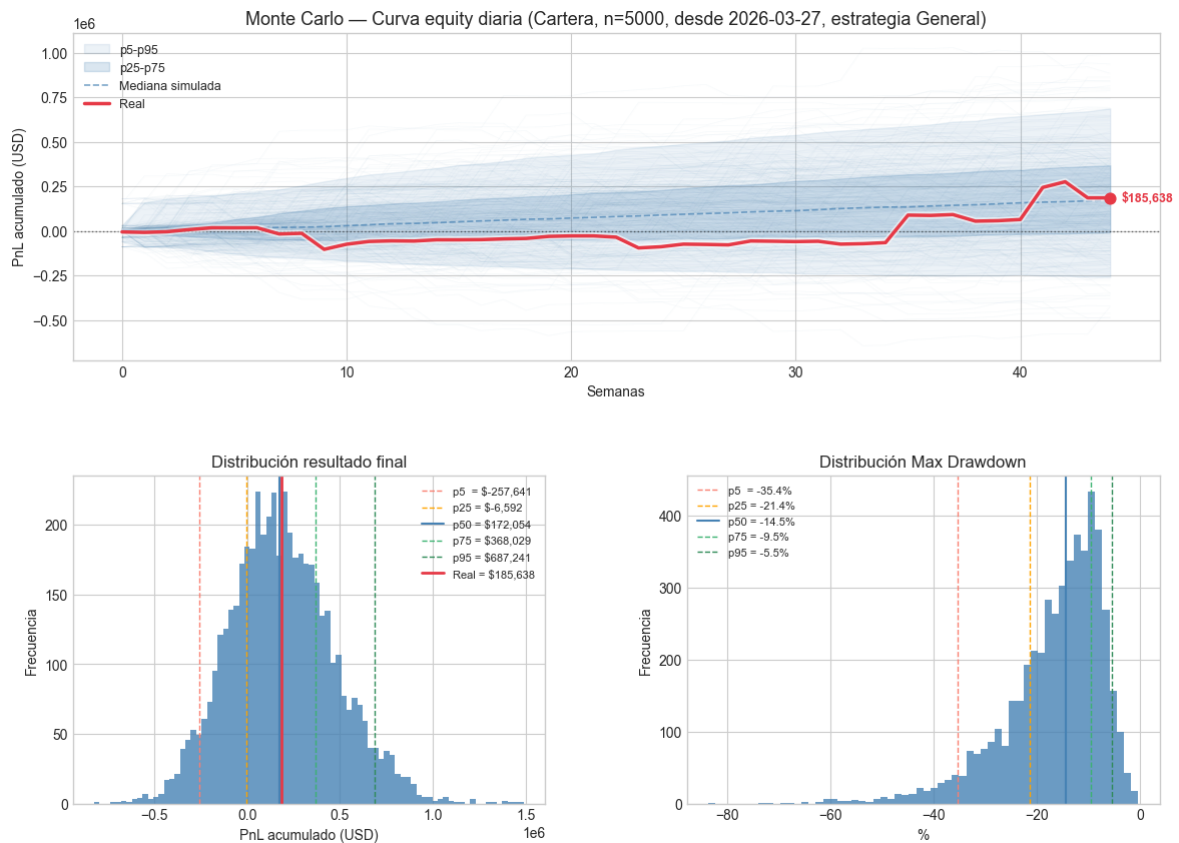


Figura 4.3: Añadir img comparativa del Monte Carlo vs Robotrader



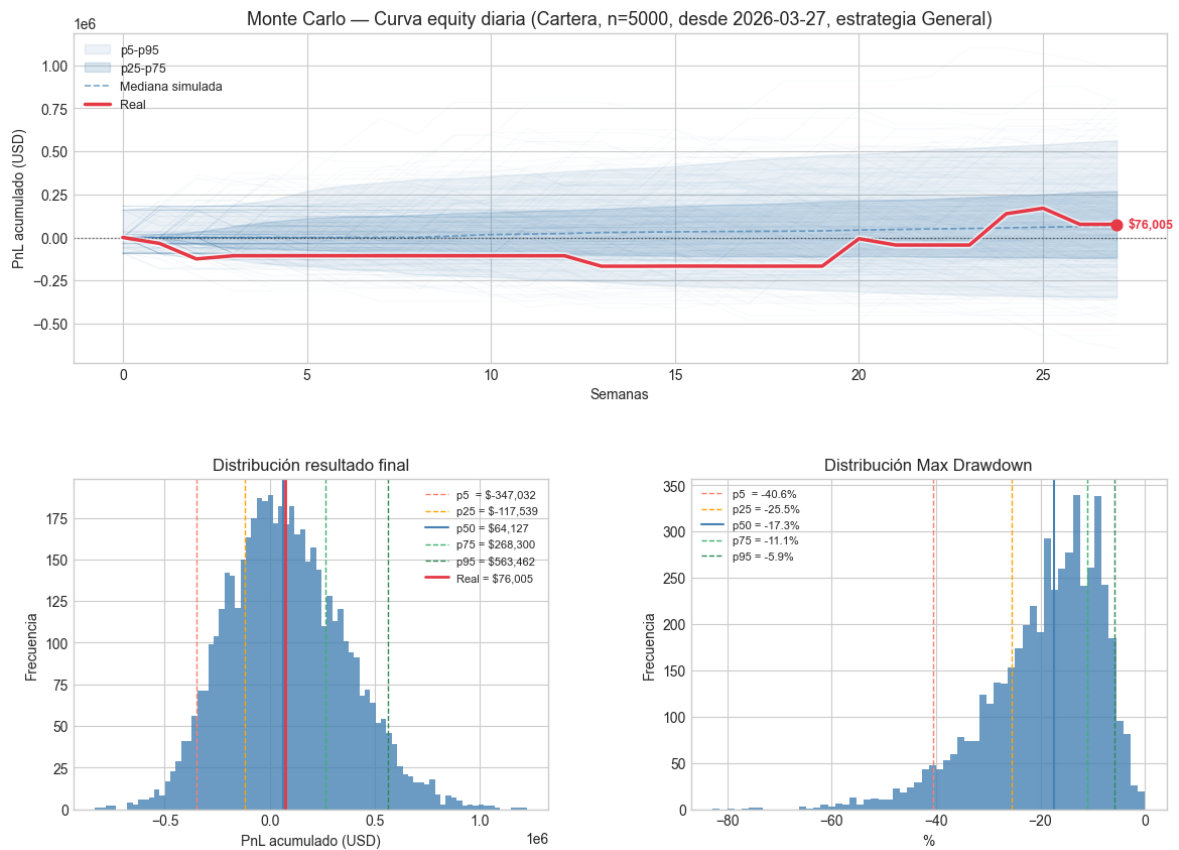


Figura 4.5: Añadir img comparativa del Monte Carlo vs Robotrader

Añadir grafica de la evolución del equity, comparativa contra Robotrader

Añadir Monte Carlo

Futuro	Capital inicial	Capital final				Ganancias mejor caso
		Normal	Scale-in	Sizing	Scale-in + Sizing	
AD	30k	No hay operaciones				0
BP	30k	Por alguna razón faltan dos features				0
CL	300k	305k	317k	306k	400k	100k
EC	40k	No hay operaciones				0
ES	350k	320k	280k	349k	250k	-1k
GC	450k	450k	465k	450k	465k	15k
MFXI	5k	4.8k	4.75k	4.5k	3.75k	-0.2k
NG	70k	70k	69k	71k	67k	1k
NQ	650k	630k	570k	650k	550k	0
YM	200k	198k	198k	200k	196k	0
ZB	50k	50k	49k	50k	48k	0
ZN	30k	25k	24k	22k	0k	-5k
ZS	50k	300k	300k	80k	300k	250k

Tabla 4.1: Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-01-20 hasta 2026-06-01

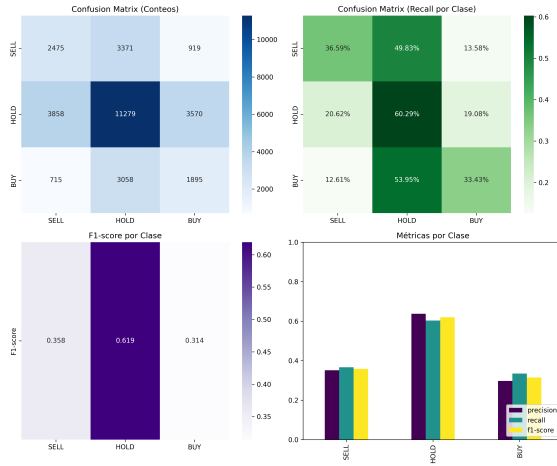
Como capital inicial he usado 10 veces el margin inicial mínimo para abrir una posición

Futuro	Capital inicial	Ganancias mejor caso	Max DD	Ratio
CL	300k	0		
ES	350k	-1k		
GC	450k	8510	6730	1.26
MFXI	5k	-0.2k		
NG	70k	904	2116.23	0.43
NQ	650k	17385	0	No aplica
YM	200k	6825	2375.0	2.87
ZB	50k	0		
ZN	30k	-2.5k		
ZS	50k	-2k		

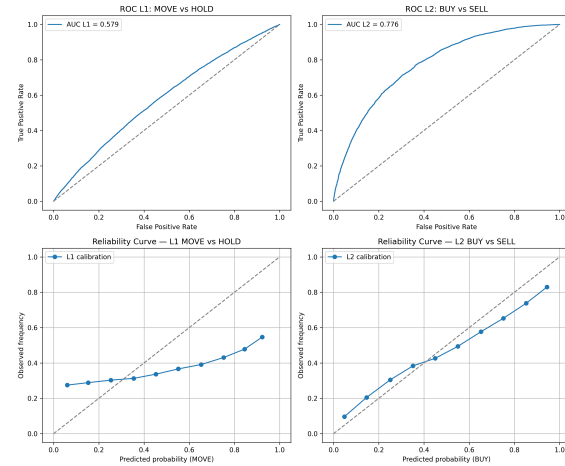
Tabla 4.2: Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-03-27 hasta 2026-06-01, aunque Robotrader empezó el 6 para tener datos para las primeras predicciones

En ganancias absolutas séptimo/noveno en la competición de 58 personas.

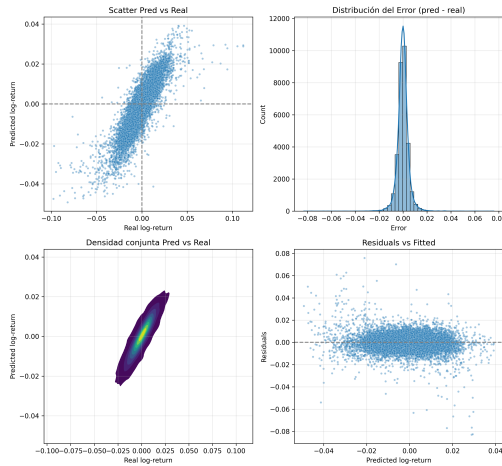
4.1.1 Análisis de resultados de entrenamiento



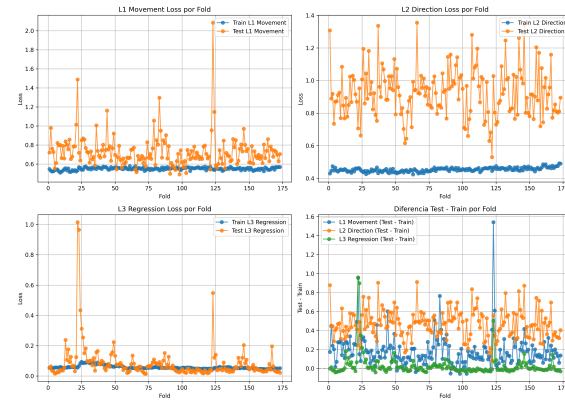
(a) Classification report



(b) ROC y reliability curve

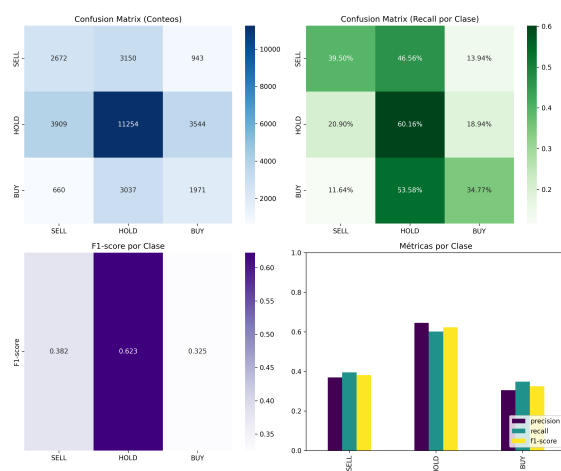


(c) MAE y regresión

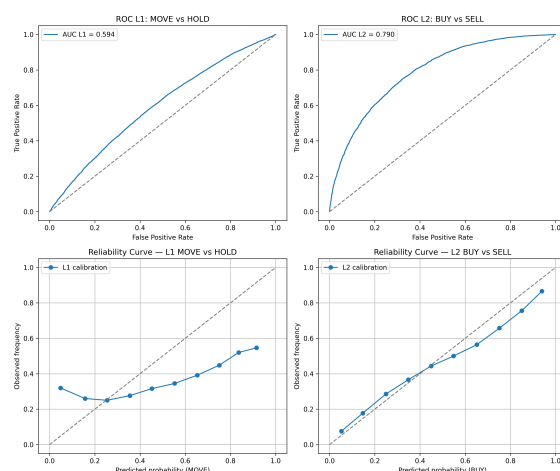


(d) Loss

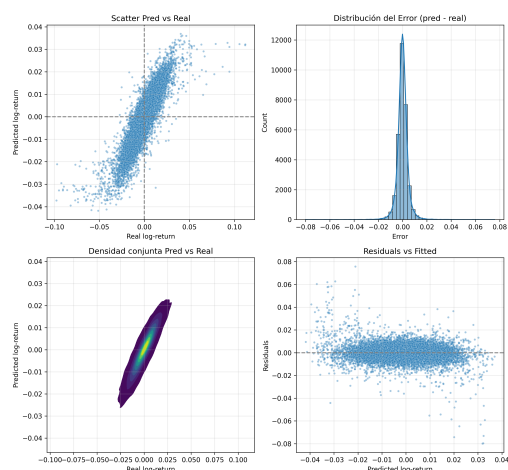
Figura 4.6: Resultados obtenidos por el modelo LSTM.



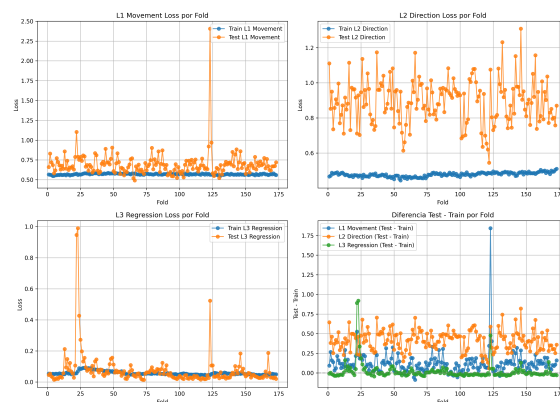
(a) Classification report



(b) ROC y reliability curve



(c) MAE y regresión



(d) Loss

Figura 4.7: Resultados obtenidos por el modelo GRU.

Añadir curva auc roc para las dos capas de clasificación

Añadir curva loss por fold

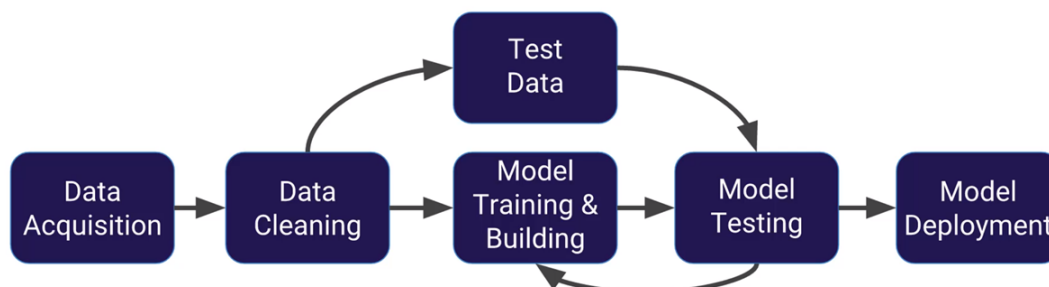


Figura 4.8: Añadir img con evolución de las métricas dependiendo de los umbrales

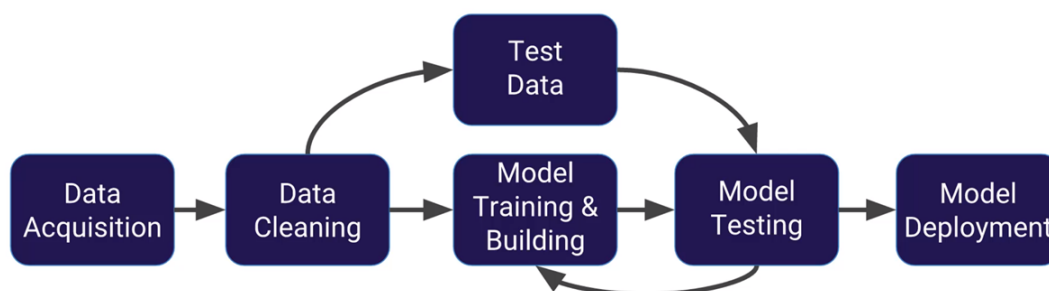


Figura 4.9: Añadir img con una simulación de Monte Carlo con los percentiles

4.1.2 Análisis de resultados de optimización, Backtest de la cartera y Simulación de Monte Carlo

Una vez obtenidas las señales optimizadas para cada futuro, se realiza un análisis conjunto del comportamiento de la cartera completa. El objetivo es evaluar si la combinación de activos mejora el rendimiento respecto a operar únicamente un activo individual, así como estudiar la robustez del sistema bajo diferentes secuencias de resultados.

Para ello se lleva a cabo un nuevo backtest a nivel de cartera. En este proceso se cargan todos los *trades* generados por cada futuro utilizando la estrategia operativa óptima obtenida previamente. A continuación, se construye un único *DataFrame* que agrupa todas las operaciones por fecha, lo que permite simular la evolución conjunta del capital.

La simulación se realiza sobre una cuenta con un capital inicial de 1 000 000 USD, lo que permite incorporar restricciones realistas de riesgo y liquidez. En particular, se tienen en cuenta:

- los márgenes iniciales y de mantenimiento de cada futuro,
- el valor nocional de cada contrato,
- el porcentaje máximo de capital que puede arriesgarse por operación,
- la exposición total máxima permitida, fijada en un 25 % del capital inicial.

Las predicciones se ordenan por nivel de confianza total y se procesan secuencialmente, verificando en cada caso que la operación no exceda los límites de margen ni las restricciones de exposición. Solo las operaciones que cumplen estas condiciones se ejecutan en la simulación.

Además del backtest, se realiza una simulación de Monte Carlo adicional para evaluar la dispersión de las métricas de la cartera completa. Para ello se agrupan los PnL diarios de todos los activos y se aplica un *overlapping block bootstrap*, remuestreando bloques solapados de la serie diaria de beneficios. Este enfoque permite preservar parte de la estructura temporal y generar múltiples trayectorias alternativas del rendimiento de la cartera.

El análisis conjunto del backtest y de la simulación de Monte Carlo permite comparar el rendimiento de la cartera diversificada frente a la operativa de un único activo, así como evaluar la estabilidad del sistema bajo diferentes escenarios de mercado.

Conclusiones y líneas futuras

5.1 Conclusiones

Como conclusiones finales de este proyecto, estoy muy satisfecho con lo que he desarrollado en el transcurso de estos meses. Además, viendo los resultados obtenidos se puede concluir que el sistema ha funcionado como se esperaba.

He podido aplicar los conocimientos que he adquirido durante estos 4 años de carrera aunque hayan habido teoría de asignaturas que me gustaría haber podido aplicar que se han quedado fuera por falta de tiempo. Aún así también he podido ampliar mis conocimientos sobre diseño de redes neuronales.

5.2 Líneas futuras

Preguntar si demasiadas líneas futuras demuestra poco o mucho interés en el trabajo

Por limitaciones del tiempo dejo pendiente varias mejoras e implementaciones nuevas

Lo primero y de lo que se ha hablado en el desarrollo, es la implementación de modelos no supervisados en diferentes puntos del pipeline, por una parte como feature extra, por otra parte como segmentador de regímenes de mercado y por último como agrupador de futuros.

Además, también me gustaría añadir el despliegue de MLflow y Airflow ya que son partes esenciales del trabajo como ingeniero y científico de datos, aportarían mucho valor.

A parte de estas implementaciones que se han comentado en los apartados de desarrollo también existen otras cosas que me gustaría añadir a futuro.

Por una parte añadir una interfaz gráfica independiente de grafana para tener personalización total. Durante la carrera he tenido asignaturas de desarrollo Front-End que me gustaría aplicar, serviría para mostrar más información a parte de la que permite grafana además de incluir la posibilidad de interactuar con los sistemas, poder parar o arrancar el bot, decidir la lista de activos a operar, etc.

En relación con lo anterior, me gustaría añadir en esa misma página un chatbot usando RASA o un LLM para responder preguntas a partir de toda la información disponible. RASA es mucho más sencillo pero un LLM permite mayor variedad de respuestas además de que a futuro se podría implementar una API MCP (Model Context Protocol)

Más relacionado con la carrera sería aplicar conceptos como la entropía de Shannon, las cadenas de Markov junto con las FSM (Finite State Machine) y el análisis de sentimientos usando NLP con BERT y MarIA

Entropía de Shannon y teoría de la información

La entropía mide la incertidumbre del mercado

Cadenas de Markov y FSM

Modelan la probabilidad de transición entre estados de mercado, las FSM estructuran el comportamiento del bot

Análisis de sentimientos

Permiten incluir información textual de noticias y redes sociales como adición a indicadores cuantitativos.

Además de todo lo ya mencionado a continuación quiero listar una serie de mejoras que se me han ido ocurriendo al final del trabajo y durante mis jornadas estando de prácticas

Great Expectations

Como complemento a la validación manual de los datos

Engine/pipeline

Durante mis prácticas pude observar cómo trabajaban en un pipeline real de datos en un banco, se organizaban como un pipeline, cada paso es una función única y todas comparten un contexto común

Otros productos financieros

Aunque ya se justificó por qué le decidió elegir los futuros aún así se puede experimentar con otros productos como las acciones, opciones, etc. Además, se podría optar a usar los datos de los otros productos como features

Mayor nivel de profundidad de datos

Aunque los datos históricos son solo OHLCV, IB tiene más información, de la que destaca es la diferenciación de los precios de bid y ask

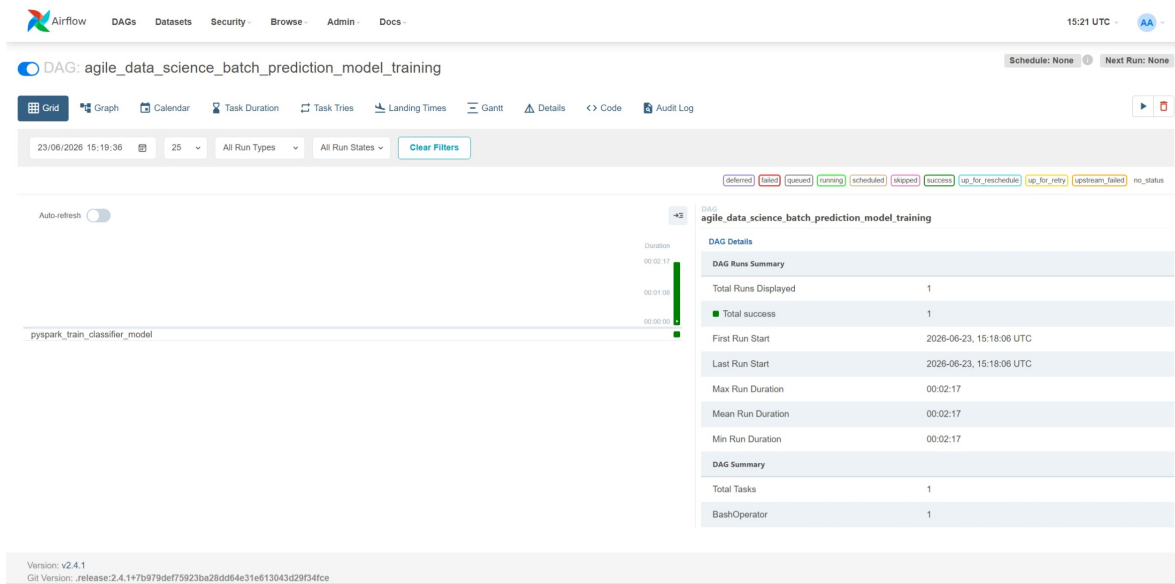


Figura 5.1: Añadir img con algunos plots de las características más usadas como modelos solos

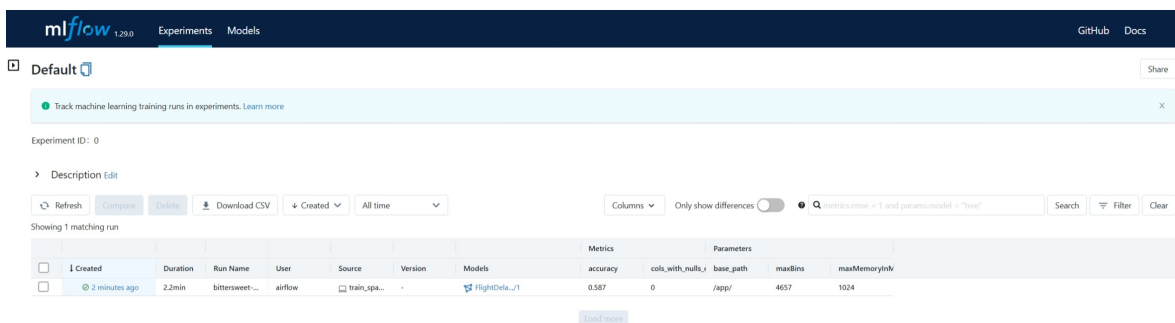


Figura 5.2: Añadir img con algunos plots de las características más usadas como modelos solos

5.2.1 Aprendizaje no supervisado

Además de usar modelos de aprendizaje supervisado tenía la idea de usar modelos no supervisados para varias tareas.

En primer lugar, para agrupar los diferentes activos en diferentes grupos. En vez de realizar agrupaciones arbitrarias, por ejemplo, todas las monedas juntas, dejo que el sistema decida cuales son los activos más parecidos. De este modo podría generar más características derivadas de la comparación de los activos, basandome en el concepto de *Pairs Trading* donde se comparan un par de activos que se saben que están correlados y cointegrados, y si uno de ellos sube más que el otro se abre una posición en corto y si uno de ellos baja más que el otro se abre una posición en largo.

El problema que le encuentro a este primer caso de uso es la disparidad de datos, por una parte, los datos históricos proporcionados no son de las mismas fechas, esto se soluciona fácilmente al recortar los datos más antiguos ya que además podrían causar problemas a los modelos. Pero donde existen más problemas es el tema de los horarios de las diferentes bolsas, la idea sería usar la función de rellenado hacia adelante para los momentos donde faltasen datos de alguno de los futuros. Otra opción sería comparar cada futuro con el centroide (el centro de cada cluster) del cluster al que pertenece cada futuro y el resto de centroides, de este modo se puede evitar problemas por la disparidad del número de futuros en cada cluster.

Además de este problema está la necesidad de tomar la decisión de qué tipo de modelo no supervisado usar, si uno donde se predefine el número de clusters o uno donde el propio modelo decide el número (Decidir entre modelos paramétricos (k fijo) o modelos no paramétricos (k variable))

Y el problema principal sería sobre qué agrupar qué características usar para la agrupación.

El segundo uso es el de segmentar los regímenes de mercado. Con esta información hay dos opciones de acción, por una parte, usarlo como una característica más para complementar al resto, aunque puede no ser tan útil al usar modelos de aprendizaje supervisado profundo que al estar diseñados para manejar series temporales pueden inferir los regímenes de mercado por si solos. La segunda opción es usar la diferencia de regímenes para entrenar modelos dedicados para cada régimen consiguiendo modelos más especializados en cada régimen. Este uso tiene los mismos problemas que el primero.

5.2.2 Pyomo

Una vez que tuve el sistema funcionando paso a implementar un optimizador usando Pyomo, para solvertar algunas limitaciones que tiene el método manual, principalmente el hecho de que una predicción con menor confianza pueda generar mayor beneficio.

Explicar qué es Pyomo

Para ello sigo los siguiente paso para una optimización de un objetivo, en este caso maximizar el *Sharpe Ratio*

1. Definir variables, conjunto de activos,
2. Definir parámetros, retorno esperado, volatilidad, matriz de covarianza, capital disponible, límite de exposición, restricciones de riesgo
3. Definir variable de decisión, abrir o no una posición
4. Definir restricciones, suma de pesos, límite de exposición, restricciones de riesgo, restricciones operativas
5. Definir función objetivo, Sharpe Ratio
6. Seleccionar solver
7. Interpretar solución

Como mecanismo adicional de control, los modelos se actualizan semanalmente. Cada semana se reentrenan o afinan los modelos existentes y se comparan con los anteriores para seleccionar aquellos que mejor predicen los datos recientes. Este proceso garantiza que el sistema se mantenga adaptado a posibles cambios de régimen o variaciones en la estructura del mercado. La orquestación completa de este procedimiento se detalla en una sección posterior.

En caso de haber continuado con el enfoque de agrupación mediante aprendizaje no supervisado, también sería posible definir límites de exposición por grupo. Esto permitiría evitar situaciones en las que múltiples señales positivas pertenecen a un mismo cluster y, si dicho grupo estuviera mal modelado, se concentraría demasiado riesgo en un único patrón de comportamiento.

5.2.3 Orquestación y MLOps

Tanto Airflow como MLflow, ver si hay tiempo para implementarlos

Una vez desplegado un sistema robusto que cuenta con un autómata funcional y con alertas tanto de seguridad como para mostrar información, decido implementar el uso de Airflow como orquestrador.

En general el uso de Airflow está muy ligado al uso de MLflow ya que será el uso principal de esta herramienta por mi parte.

En MLflow

Apache Airflow es un orquestador de flujos de trabajo que permite programar, monitorizar y ejecutar tareas complejas de forma secuencial o paralela mediante DAGs (Directed Acyclic Graphs).

MLflow es una plataforma para gestionar el ciclo de vida de modelos de Machine Learning: tracking, versionado, artefactos, modelos, comparaciones y despliegue.

Los usos principales de MLflow para mí son el registro de experimentos, que en vez de realizar los entrenamientos manualmente se realicen aquí permitiendo guardar toda la información de cada entrenamiento, permite comparar diferentes modelos y de este modo decidir si es hora de cambiar un modelo por una versión actualizada o un nuevo modelo, permite llevar un registro de los modelos que se están utilizando, de los usados o los que están probándose. Facilita la búsqueda de parámetros de los modelos.

Airflow por su parte sirve para orquestar muchas de las funcionalidades de MLflow, orquestación del ciclo de vida del modelo, entrenamientos semanales, fine tuning incremental con los datos, evaluación automática de nuevos modelos, comparación contra el modelo actual, promoción del mejor modelo a producción, también permite automatización del bot, que despliegue el bot los sábados por la tarde y lo pare el viernes por la tarde, reinicios controlados.

El problema principal es que Airflow es muy pesado y consume mucha RAM por lo que algunas de las funcionalidades no se pueden realizar aún así se podría activar manualmente los fines de semana para poder realizar las tareas relacionadas con el ciclo de vida de los modelos.

Aspectos éticos, económicos, sociales y ambientales

6.1 Introducción

Desde el punto de vista económico el sistema tiene influencia en la reducción de costes necesarios para operar una cartera de activos, permitiendo a profesionales autónomos o quienes lo toman como un hobby. En el ámbito ético y social, . Y por último, con respecto al impacto ambiental, este sistema no tiene casi impacto al poder ejecutarse en local en hardware particular no profesional.

6.2 Descripción de impactos relevantes relacionados con el proyecto

Impacto económico

1

Impacto ético y social

2

Impacto ambiental

3

6.3 Análisis detallado de alguno de los principales impactos

El impacto mayoritario de este proyecto se centra en el tema económico

6.4 Conclusiones

Como conclusión final

Presupuesto económico

COSTE DE MANO DE OBRA (coste directo)			Horas	Precio/hora	Total
			360	25 €	9.000 €
COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total	
Ordenador personal (Software incluido)	1.500,00 €	6	5	150,00 €	
COSTE TOTAL DE RECURSOS MATERIALES					150,00 €
GASTOS GENERALES (costes indirectos)	15 %	sobre CD			1372,50 €
BENEFICIO INDUSTRIAL	6 %	sobre CD+CI			631,35 €
SUBTOTAL PRESUPUESTO					11.153,85 €
IVA APLICABLE				21 %	2.342,3085 €
TOTAL PRESUPUESTO					13.496,1585 €

Bibliografía

- [1] F. Love, “Nntikz: Tikz diagrams for deep learning and neural networks,” 2024.
- [2] I. Trullàs, *Trading algorítmico con Python: Desarrolla tus habilidades en el Trading Algorítmico. Con ejemplos e integración a MetaTrader5*. Amazon Kindle Direct Publishing, 2024. Kindle edition.
- [3] D. E. Bowden, *Safety in the Market: The Smarter Starter Pack*. Queensland, Australia: Safety in the Market, third revised edition ed., 1997. ASIN: B000LYUHJE.
- [4] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [5] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.

Anexos

A Resultados de los entrenamientos

B Resultados de los backtest individuales

C Repositorio de GitHub con el código

El código completo desarrollado para este trabajo se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/LijunZhou000/ML-Algorithmic-Trading>